CLOUD NATİVE MİMARİ, MİKROSERVİSLER ve SERVERLESS

İÇİNDEKİLER

CLOUD NATİVE MİMARİ, MİKROSERVİSLER ve SERVERLESS 1

Tablolar Listesi 2

Şekiller Listesi 2

1. Dersin Amacı ve Öğrenme Çıktıları 3
   - 1.1. Dersin amacı 3
   - 1.2. Öğrenme Çıktıları 3
2. Bulutta Uygulama Mimarileri: Cloud Native, SOA ve Mikroservisler 3
   - 2.1. SOA ve Hizmet Sözleşmeleri 5
   - 2.2. Monolitik Uygulama, Modüler Monolit ve Mikroservisler 6
   - 2.3. Hizmet Sınırları ve Veri Sahipliği 8
3. Durum Yönetimi, Hizmet Arayüzleri ve Hizmetlere Erişim 10
   - 3.1. Stateless ve Stateful Bileşenler 10
   - 3.2. REST ve Temel API Tasarımı 11
   - 3.3. API Sözleşmesi, Doğrulama ve İşlem Durumu 13
   - 3.4. API Ağ Geçidi, Hizmet Keşfi ve Yük Dengeleme 15
4. Hizmetler Arası İletişim, Mesajlaşma ve Olay Güdümlü Mimari 17
   - 4.1. Eşzamanlı ve Eşzamansız İletişim 17
   - 4.2. İş Kuyruğu ve Yayınla–Abone Ol Modeli 19
   - 4.3. Olay, Komut ve Olay Güdümlü Akış 20
   - 4.4. Kısmi Hata, Tekrar Teslim ve İdempotentlik 22
   - 4.5. Nihai Tutarlılık ve Başarısız İşlerin Yönetimi 24
5. Serverless ve Fonksiyon Tabanlı Çalıştırma 25
   - 5.1. Serverless Yaklaşım ve FaaS 26
   - 5.2. Tetikleyiciler ve Fonksiyonun Çalışma Akışı 27
   - 5.3. Başlangıç Gecikmesi, Sınırlar ve Model Seçimi 30
6. Bulut Sağlayıcı Örnekleri ve Bütünleşik Uygulama Senaryosu 32
   - 6.1. AWS, Azure ve Google Cloud Kavram Eşleştirmesi 32
   - 6.2. Etkinlik Kayıt Uygulamasının Gereksinimleri ve Mimarisi 33
   - 6.3. Aşamalı Yerel Uygulama 36
   - 6.4. İsteğe Bağlı Bulut Uygulaması ve Değerlendirme 38
7. Bölüm Özeti 39

Kaynakça 40
Hafta 7 – Çalışma Soruları 42

Tablolar Listesi

Tablo 1. Uygulama mimarisi, paketleme ve çalıştırma kararlarının ayrımı. 4
Tablo 2. Etkinlik kaydı oluşturma hizmetinin örnek sözleşme unsurları. 5
Tablo 3. Monolitik uygulama, modüler monolit ve mikroservislerin karşılaştırılması. 7
Tablo 4. Temel HTTP yöntemleri ve etkinlik uygulamasındaki örnek kullanımları. 12
Tablo 5. API ağ geçidi, hizmet keşfi ve yük dengelemenin temel görevleri. 16
Tablo 6. Eşzamanlı ve eşzamansız iletişimin karşılaştırılması. 18
Tablo 7. Olay ve komut mesajlarının anlam bakımından karşılaştırılması. 21
Tablo 8. Yönetilen FaaS modelinde temel sorumlulukların paylaşılması. 27
Tablo 9. FaaS ve konteyner tabanlı çalıştırma için seçim ölçütleri. 31
Tablo 10. Temel mimari ihtiyaçların bulut sağlayıcılarındaki örnek karşılıkları. 32
Tablo 11. Etkinlik kayıt uygulamasının bileşenleri ve veri sorumlulukları. 34
Tablo 12. Yerel uygulama deneyleri ve beklenen sonuçlar. 37

Şekiller Listesi

Şekil 1. Hizmet sorumlulukları ve veri sahipliği sınırları. 9
Şekil 2. İş kuyruğunda yük paylaşımı ve yayınla–abone ol modelinde bağımsız dağıtım. 20
Şekil 3. Tetiklenen fonksiyonun çalıştırılması ve kalıcı verilerle etkileşimi. 29
Şekil 4. Etkinlik kaydının oluşturulması ve bildirim işinin ayrı yürütülmesi. 35

1. Dersin Amacı ve Öğrenme Çıktıları

1.1. Dersin amacı

Önceki haftalarda bulut uygulamalarının ağ ve veri gereksinimleri, konteynerlerle paketlenmesi ve Kubernetes üzerinde çalıştırılması ele alınmıştır. Bu hafta, uygulamaların bulut ortamına uygun biçimde tasarlanmasına odaklanılmaktadır.

Amaç; cloud native yaklaşımı çerçevesinde hizmet odaklı mimariyi, mikroservisleri, hizmetler arası iletişimi ve serverless çalıştırma modelini anlamaktır. Uygulamanın hizmetlere ayrılması, durumun yönetilmesi ve iş yüküne uygun iletişim ve çalıştırma yöntemlerinin seçilmesi, bulutun esneklik ve yönetilebilirlik olanaklarıyla ilişkilendirilecektir. Bu kavramsal temel, küçük bir uygulama üzerinde yapılacak çalışmalarla pekiştirilecektir.

1.2. Öğrenme Çıktıları

Bu haftayı ve temel uygulama çalışmalarını tamamlayan öğrenci:

1. Monolitik ve mikroservis mimarilerini karşılaştırır; küçük bir bulut uygulamasında hizmet sınırlarını ve veri sorumluluklarını açıklar.
2. Hizmet odaklı mimariyi hizmet sözleşmeleriyle ilişkilendirir; hazır bir hizmet arayüzünü kullanır ve üzerinde küçük bir değişiklik yapar.
3. Durumsuz ve durum bilgili bileşenleri ayırt eder; verinin tutulduğu yerin uygulamanın yeniden başlatılması ve çoğaltılması üzerindeki etkisini açıklar.
4. İki hizmeti ayrı çalıştırır; doğrudan çağrı ile kuyruk üzerinden işleme arasındaki farkı ve aynı işin tekrar gönderilmesinin sonuçlarını gözlemler.
5. Fonksiyon tabanlı çalıştırma yaklaşımını açıklar; basit bir fonksiyonu yerelde çalıştırır ve iş yükü için fonksiyon veya konteyner seçimini gerekçelendirir.

İsteğe bağlı bulut uygulamasını tamamlayan öğrenci ayrıca, seçtiği sağlayıcıda bir fonksiyonu dağıtır, tetikleyicisini yapılandırır ve çalışma sonucunu doğrular.

2. Bulutta Uygulama Mimarileri: Cloud Native, SOA ve Mikroservisler

2.1. Bulutta Çalışmak ve Buluta Uygun Tasarlamak

Bir uygulamanın buluttaki sanal makinede veya konteynerde çalıştırılması, bulut kaynaklarını kullandığını gösterir. Ancak bu uygulamanın değişken iş yüküne, çalışma örneklerinin yenilenmesine ve bileşenler arasındaki hatalara uygun tasarlandığı anlamına gelmez. Buluta taşıma, uygulamanın çalıştığı ortamı değiştirirken; buluta uygun tasarım, uygulamanın bu ortamda nasıl davranacağını da ele alır.

Cloud native yaklaşımının temelinde, uygulamaların bulut ortamlarında ölçeklenebilir ve tekrarlanabilir biçimde geliştirilmesi, dağıtılması ve yönetilmesi bulunur. CNCF — Cloud Native Computing Foundation (Bulut Yerel Bilişim Vakfı), bu yaklaşımda gevşek bağlı,

dayanıklı, yönetilebilir ve gözlemlenebilir sistemleri vurgular. Konteynerler, mikroservisler ve serverless bu yaklaşımı destekleyen teknolojiler arasındadır; bütün uygulamaların bunların tamamını kullanması gerekmez. [1]

Buluta uygun bir tasarımda aşağıdaki sorular önem kazanır:

- **Yük arttığında ne olur?** Yeni uygulama örnekleri eklendiğinde istekler bu örnekler arasında paylaşılabilmelidir. Gerekli kullanıcı veya işlem bilgisinin yalnızca tek örneğin belleğinde bulunması, bu paylaşımı zorlaştırabilir.
- **Bir çalışma örneği yeniden oluşturulduğunda ne olur?** Kaybolmaması gereken iş verisi, örneğin geçici yaşam döngüsünden bağımsız korunmalıdır.
- **Bir bileşen geciktiğinde ne olur?** Bildirim gönderme gibi bir işin gecikmesi, temel kayıt işlemini gereksiz yere bekletmemelidir.
- **Bir işlev değiştirildiğinde ne olur?** Bileşen sınırları ve arayüzleri, değişikliğin diğer işlevler üzerindeki etkisini sınırlayacak biçimde belirlenmelidir.

Örneğin bir etkinlik kayıt uygulamasında kayıt ve bildirim işlemleri aynı istek içinde yürütülsün. Bildirim sağlayıcısı yavaşladığında kullanıcı, kaydı tamamlanmış olsa bile yanıt bekleyebilir. Bildirim işi ayrı bir arka plan akışına aktarılırsa kayıt işlemi kullanıcıya daha erken yanıt verebilir. Bunun karşılığında bildirimin tamamlanma durumunu izlemek ve başarısız işleri ele almak gerekir. Dolayısıyla buluta uygun tasarım, yalnızca bileşenleri ayırmayı değil, bu ayrımın yararlarını ve getirdiği sorumlulukları birlikte değerlendirmeyi gerektirir.

Önceki haftalarda öğrenilen konteyner ve Kubernetes kavramları, uygulamayı paketleme ve çalıştırma temelini sağlar. Bu haftanın mimari kararları ise farklı soruları yanıtlar. Bu ayrım Tablo 1'de gösterilmiştir.

Tablo 1. Uygulama mimarisi, paketleme ve çalıştırma kararlarının ayrımı.

Karar alanı	Temel soru	Örnek
Uygulama mimarisi	İşlevler nasıl ayrılacak ve haberleşecek?	Kayıt ve bildirim işlevlerinin aynı uygulamada veya ayrı hizmetlerde bulunması
Paketleme	Kod ve bağımlılıkları nasıl dağıtılacak?	Uygulamanın konteyner imajına dönüştürülmesi
Çalıştırma ve yönetim	Uygulama örnekleri nasıl yürütülecek?	Konteynerlerin Kubernetes üzerinde yönetilmesi veya bir işlevin fonksiyon olarak çalıştırılması

Bir uygulamayı konteynere dönüştürmek, onun hizmet sınırlarını kendiliğinden değiştirmez. Benzer biçimde Kubernetes kullanılması, uygulamanın mikroservis mimarisine sahip olduğunu göstermez. Buluta uygunluk değerlendirilirken kullanılan araçlarla birlikte verinin korunması, bileşenlerin bağımlılıkları ve iş yükünün davranışı incelenmelidir.

2.1. SOA ve Hizmet Sözleşmeleri

SOA — Service-Oriented Architecture (hizmet odaklı mimari), yazılım yeteneklerinin tanımlı hizmetler aracılığıyla kullanıma sunulmasına dayanan bir mimari yaklaşımdır. Bir hizmet, belirli bir işlevi gerçekleştirir ve bu işlevin başka uygulamalar tarafından nasıl kullanılacağını açıklar. SOA'nın temelinde, hizmeti kullanan sistemin, hizmeti sunan sistemin iç işleyişini bilmeden, onun sunduğu hizmetlerden yararlanabilmesi yatar. [2]

Örneğin bir üniversitenin etkinlik kayıt uygulaması, katılımcının öğrenci olup olmadığını doğrulamak isteyebilir. Bunun için öğrenci bilgi sisteminin veritabanına doğrudan bağlanmak yerine, bu sistemin sunduğu öğrenci doğrulama hizmetini kullanabilir. Kayıt uygulaması gerekli bilgiyi gönderir ve doğrulama sonucunu alır. Öğrenci bilgilerinin hangi tablolarda tutulduğunu veya doğrulamanın hangi programlama diliyle gerçekleştirildiğini bilmesi gerekmez. Bu etkileşimde iki temel rol vardır:

- **Hizmet sağlayıcısı:** Yeteneği sunan ve işlemi gerçekleştiren bileşendir.
- **Hizmet tüketicisi:** Sunulan yeteneği kendi iş akışında kullanan bileşendir.

Aynı uygulama farklı etkileşimlerde her iki rolü de üstlenebilir. Etkinlik kayıt uygulaması öğrenci doğrulama hizmetini tüketirken, başka uygulamalara kayıt sorgulama hizmeti sunabilir.

Hizmet sözleşmesi

Bir hizmetin kullanılabilmesi için yalnızca erişim adresinin bilinmesi yeterli değildir. Tüketici, hangi bilgileri göndereceğini ve aldığı sonucu nasıl yorumlayacağını da bilmelidir. *Service contract (hizmet sözleşmesi)*, hizmetin sunduğu işlemleri ve etkileşim kurallarını tanımlar. Bu bölümde sözleşme, hukuki bir anlaşmadan ziyade yazılımlar arasındaki kullanım kuralları anlamında kullanılmaktadır.

Tablo 2, etkinlik kaydı oluşturma hizmeti için örnek bir sözleşmenin temel unsurlarını göstermektedir.

Tablo 2. Etkinlik kaydı oluşturma hizmetinin örnek sözleşme unsurları.

Unsur	Açıklama	Örnek
İşlem	Hizmetin sunduğu yetenek	Katılımcıyı etkinliğe kaydetme
Girdiler	Gönderilmesi gereken bilgiler	Etkinlik kimliği ve katılımcı kimliği
Kurallar	İşlemin kabul koşulları	Etkinlik açık olmalı; kontenjan bulunmalı
Başarılı sonuç	İşlem gerçekleştiğinde dönen bilgi	Kayıt kimliği ve kayıt durumu
Hata sonucu	İşlem gerçekleştirilemediğinde dönen bilgi	Kontenjan dolu veya etkinlik bulunamadı
Tekrar davranışı	Aynı isteğin yeniden gönderilmesinde uygulanacak kural	Aynı katılımcı için ikinci kayıt oluşturulmaması

Sözleşme, tüketicinin ihtiyaç duyduğu davranışı açıklamalı; sağlayıcının iç uygulama ayrıntılarını açığa çıkarmamalıdır. Örneğin kayıt sonucunda "kayıt kimliği" döndürmek anlamlıdır; bu kaydın veritabanındaki fiziksel konumunu bildirmek gerekli değildir.

Gevşek bağlılık ve buluttaki kullanım

Loose coupling (gevşek bağlılık), bileşenlerin birbirlerinin iç ayrıntılarına mümkün olduğunca az bağımlı olmasıdır. Öğrenci doğrulama hizmetinin iç veri yapısı değiştiğinde dış sözleşme korunabiliyorsa, etkinlik kayıt uygulamasının da değiştirilmesi gerekmeyebilir. Buna karşılık yanıt alanlarının anlamının değiştirilmesi, tüketicileri etkileyebilir. Dolayısıyla gevşek bağlılık, bağımlılıkların tamamen ortadan kalkması değil, açık arayüzlerle sınırlandırılmasıdır.

Bulut ortamında bu yaklaşım, farklı yerlerde çalışan uygulamaların bütünleşmesini destekler. Öğrenci bilgi sistemi kurum içinde, etkinlik uygulaması bulutta bulunabilir. Aralarındaki hizmet arayüzü, yetkili erişim ve ağ bağlantısı sağlandığında bu sistemlerin birlikte çalışmasına olanak verir. Ancak arayüz kullanılması ağ gecikmesini veya hizmet kesintisini ortadan kaldırmaz; uygulamanın bu durumları da ele alması gerekir.

SOA, belirli bir ürünün kurulmasını veya tek bir iletişim protokolünün kullanılmasını zorunlu kılmaz. Hizmet sözleşmesinde tanımlanan etkileşim, farklı teknolojilerle gerçekleştirilebilir. Örneğin kayıt uygulaması, öğrenci doğrulama hizmetine *SOAP — Simple Object Access Protocol* mesajı göndererek doğrulama isteyebilir. Burada SOA, doğrulama yeteneğinin bir hizmet olarak sunulmasını; SOAP ise bu hizmetle mesaj alışverişinin nasıl gerçekleştirileceğini belirler. Aynı hizmet, REST yaklaşımına uygun bir arayüzle de sunulabilir. Bu nedenle önce hizmetin sunduğu yetenek ve sözleşmesi, ardından iletişimde kullanılacak teknoloji belirlenmelidir.

2.2. Monolitik Uygulama, Modüler Monolit ve Mikroservisler

Bir uygulamanın işlevleri, farklı biçimlerde düzenlenip dağıtılabilir. Bu mimari seçim, bulutta hangi bileşenlerin birlikte güncelleneceğini, çoğaltılacağını ve yönetileceğini etkiler. Etkinlik yönetimi, katılımcı kaydı ve bildirim işlevlerinden oluşan bir uygulama üzerinden üç yaklaşım karşılaştırılabilir.

Monolitik uygulama

Monolithic application (monolitik uygulama), temel işlevlerin tek bir dağıtım birimi içinde sunulduğu uygulamadır. Etkinlik yönetimi, kayıt ve bildirim kodları farklı sınıflarda veya katmanlarda bulunabilir; ancak uygulama birlikte hazırlanır ve dağıtılır. Monolitik yapı, bütün kodun tek dosyada bulunması anlamına gelmez. [3]

Örneğin bildirim metni değiştirildiğinde, değişikliği içeren uygulama paketi yeniden dağıtılır. Daha fazla isteği karşılamak için uygulamanın birden fazla örneği çalıştırılabilir; bu durumda her örnek diğer işlevlerle birlikte bildirim kodunu da içerir. Dolayısıyla monolitik uygulamalar da bulutta çoğaltılabilir. [3]

Modüler monolit

Modular monolith (modüler monolit), tek dağıtım birimini korurken uygulama içinde belirgin modül sınırları oluşturur. Etkinlik, kayıt ve bildirim modüllerinin görevleri ayrılır; birbirlerinin iç ayrıntılarına doğrudan erişmeleri sınırlandırılır.

Örneğin kayıt modülü, bildirim kodunun iç işleyişini bilmek yerine tanımlı bir bildirim arayüzünü kullanabilir. Bu düzen, kodun anlaşılmasını ve değiştirilmesini kolaylaştırır. Bununla birlikte modüller aynı uygulama paketi içinde dağıtıldığı için yalnızca bildirim modülünü ayrı güncellemek veya çoğaltmak genellikle mümkün değildir.

Küçük bir ekip için bu yaklaşım, sorumlulukları ayırırken ayrı hizmetlerin ağ iletişimi ve işletim yükünü başlangıçta üstlenmemeyi sağlayabilir.

Mikroservis mimarisi

Microservices (mikroservisler) yaklaşımında uygulama, belirli iş yeteneklerine odaklanan ve bağımsız dağıtılabilen hizmetlere ayrılır. Hizmetler tanımlı arayüzler veya mesajlar üzerinden haberleşir. Bu yapı, ihtiyaç duyan hizmetin diğerlerinden ayrı güncellenmesini ve ölçeklenmesini destekler. [4]

Etkinlik uygulamasında kayıt ve bildirim ayrı hizmetler olarak çalıştırılabilir. Bildirim işlerinin sayısı arttığında yalnızca bildirim hizmetinin örnekleri çoğaltılabilir. Bildirim biçiminde yapılan değişiklik de kayıt hizmetinin yeniden dağıtılmasını gerektirmeyebilir; bunun için hizmetler arasındaki sözleşmenin uyumlu kalması gerekir.

Ancak bu ayrımın bir maliyeti vardır. Süreç içindeki bir çağrı yerine ağ iletişimi kullanıldığında gecikme, bağlantı hatası ve karşı hizmetin erişilememesi gibi durumlar ortaya çıkar. Ayrıca hizmetlerin sürümleri ve verileri arasındaki uyum yönetilmelidir. Bir hizmetin arızasının diğerlerini etkilememesi, yalnızca ayrı çalıştırılmasına değil, bağımlılıkların nasıl tasarlandığına da bağlıdır. [4]. Üç yaklaşımın temel farkları Tablo 3'te özetlenmiştir.

Tablo 3. Monolitik uygulama, modüler monolit ve mikroservislerin karşılaştırılması.

Ölçüt	Monolitik uygulama	Modüler monolit	Mikroservisler
İşlevlerin düzenlenmesi	Modül sınırları değişken olabilir	Belirgin iç modüller	Ayrı hizmetler
Dağıtım	Tek birim	Tek birim	Bağımsız hizmet birimleri
İç iletişim	Genellikle süreç içi	Modül arayüzleri üzerinden süreç içi	Ağ çağrıları veya mesajlaşma
Ölçekleme	Uygulamanın bütünü	Uygulamanın bütünü	Hizmet bazında
İşletim yükü	Görece düşük	Görece düşük	Daha fazla bileşen ve bağımlılık

Buluttaki mimari seçim, uygulamanın büyüklüğü kadar ekibin kapasitesine, değişiklik sıklığına ve farklı işlevlerin yüküne de bağlıdır. Küçük bir etkinlik uygulaması modüler monolit olarak başlayabilir. Bildirim işlevi zamanla farklı bir geliştirme ve ölçekleme ihtiyacı kazanırsa ayrı hizmete dönüştürülebilir. Uygulamayı çok sayıda konteynere bölmek yerine, bağımsızlığın nerede yarar sağlayacağını belirlemek önemlidir.

2.3. Hizmet Sınırları ve Veri Sahipliği

Mikroservis tasarımında temel karar, uygulamanın kaç parçaya bölüneceğinden önce, her hizmetin hangi iş yeteneğinden sorumlu olacağıdır. Hizmet sınırı; hizmetin gerçekleştirdiği işlemleri, uyguladığı kuralları ve yönettiği verileri belirler.

Hizmetleri yalnızca teknik katmanlara göre ayırmak yerine, anlamlı iş yetenekleri etrafında düzenlemek gerekir. Örneğin "veritabanı hizmeti" ve "iş mantığı hizmeti" şeklindeki ayrım, tek bir işin tamamlanması için sürekli karşılıklı iletişim gerektirebilir. Buna karşılık "kayıt yönetimi" gibi bir yetenek, ilişkili kuralları ve işlemleri aynı hizmette toplayabilir. [5]

İş yeteneğine göre sınır belirleme

Etkinlik uygulamasında aşağıdaki sorumluluklar düşünülebilir:

- **Etkinlik Hizmeti:** Etkinliğin adı, zamanı, yeri ve programını yönetir.
- **Kayıt Hizmeti:** Katılımcı kaydı, kayıt iptali ve kontenjan kurallarını yönetir.
- **Bildirim Hizmeti:** Bildirimlerin hazırlanmasını, gönderilmesini ve gönderim sonuçlarını yönetir.

Bu ayrımda kayıt oluşturma ve iptal etme işlemleri aynı iş yeteneğinin parçalarıdır. Bunları ayrı hizmetlere bölmek, kontenjanın doğru yönetilmesi için gereksiz bağımlılıklar oluşturabilir. Benzer biçimde her sınıf, tablo veya işlem için ayrı mikroservis oluşturmak uygun bir başlangıç değildir.

Bir sınırı değerlendirirken şu sorular sorulabilir:

- Bu işlemler aynı iş kurallarına mı bağlı?
- Genellikle birlikte mi değişiyor?
- Ayrı çalıştırılmaları veya ölçeklenmeleri somut bir yarar sağlıyor mu?
- Ayrıldıklarında sürekli birbirlerini çağırmaları gerekiyor mu?

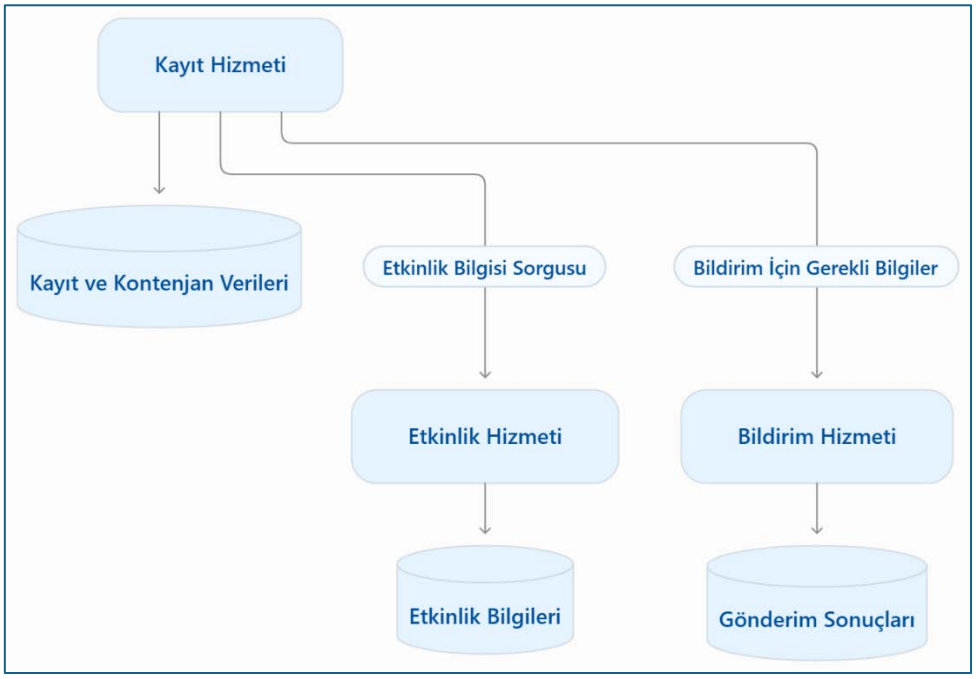
İki hizmetin küçük bir değişiklikte bile birlikte güncellenmesi gerekiyorsa, aralarındaki bağımlılık beklenenden güçlü olabilir. Hizmet sayısını artırmak yerine sınırın yeniden değerlendirilmesi gerekir.

Veri sahipliği

Data ownership (veri sahipliği), bir verinin değiştirilme kurallarından ve doğruluğundan hangi hizmetin sorumlu olduğunu belirtir. Mikroservislerde diğer hizmetlerin bu veriye doğrudan müdahale etmesi yerine, sahibinin sunduğu arayüz veya yayımladığı bilgiler üzerinden etkileşim kurulması amaçlanır. [6]

Örneğin bildirim hizmeti, katılımcıya mesaj gönderebilmek için kayıt bilgisine ihtiyaç duyabilir. Ancak bu ihtiyaç, kayıt tablosunu doğrudan değiştirme yetkisi gerektirmez. Bildirim hizmeti gerekli bilgiyi alır ve kendi gönderim sonucunu saklar. Katılımcının kaydını iptal etmek veya kontenjanı değiştirmek kayıt hizmetinin sorumluluğunda kalır.

Bu düzen Şekil 1'de gösterilmiştir. Hizmetler arasındaki oklar bilgi aktarımını, veri depolarına giden oklar ise ilgili hizmetin yönettiği veriye erişimini temsil eder.



Şekil 1. Hizmet sorumlulukları ve veri sahipliği sınırları.

Veri sahipliği, her hizmet için ayrı fiziksel veritabanı sunucusu kurulmasını zorunlu kılmaz. Aynı yönetilen veritabanı altyapısında ayrı veritabanları veya erişimi sınırlandırılmış şemalar kullanılabilir. Önemli olan, hizmetlerin birbirlerinin iç veri yapılarına doğrudan bağımlı olmaması ve değişikliklerin sorumlu hizmet üzerinden gerçekleştirilmesidir. [6]

Buluttaki tasarım kararına etkisi

Bu sınırlar, bulutta kaynakların hangi bileşene ayrılacağını da etkiler. Örneğin bildirim işleri yoğunlaştığında bildirim hizmeti ayrı çoğaltılabilir; kayıt verilerini yönetme sorumluluğu ise kayıt hizmetinde kalır. Aynı kayıt hizmetinin birden fazla çalışma örneği bulunması, farklı veri sahipleri oluştuğu anlamına gelmez. Bu örnekler aynı hizmetin kurallarını uygulayarak ortak kalıcı veriler üzerinde çalışır.

Hafta 4'te ele alınan yönetilen veri hizmetleri, verinin saklanmasını sağlar; hangi bileşenin hangi veriyi yönetmesi gerektiği ise uygulama mimarisinin kararıdır. Uygun hizmet sınırları, bulut altyapısının bağımsız çalıştırma ve ölçekleme olanaklarının anlamlı biçimde kullanılmasını destekler.

3. Durum Yönetimi, Hizmet Arayüzleri ve Hizmetlere Erişim

3.1. Stateless ve Stateful Bileşenler

Bir uygulamanın işleyişinde, sonraki işlemleri etkileyen bilgiler bulunur. Kullanıcının oturumu, devam eden kayıt işlemi veya oluşturulmuş bir katılımcı kaydı bu bilgilere örnektir. Bu bilgiler uygulamanın *durumunu* oluşturur. Bulut ortamında durumun nerede tutulduğu, uygulama örneklerinin çoğaltılmasını ve yeniden oluşturulmasını doğrudan etkiler.

Stateless (durumsuz) bir uygulama bileşeninde, sonraki isteğin doğru işlenmesi belirli bir çalışma örneğinin önceki isteklerden sakladığı yerel bilgiye bağlı değildir. *Stateful (durum bilgili)* bir bileşen ise işlemler arasında durumu korur ve sonraki işlemlerde bu durumdan yararlanır.

Buradaki ayrım, verinin bulunup bulunmaması değil, işlemin hangi bileşende korunan duruma bağımlı olduğudur. Durumsuz bir hizmet, veritabanından bilgi okuyabilir ve kalıcı kayıt oluşturabilir.

Durumsuz hizmet örneği

Etkinlik kayıt hizmetinin A ve B olmak üzere iki örneği bulunduğunu düşünelim. Kullanıcı A örneği üzerinden kayıt oluşturduktan sonra kayıt durumunu sorguladığında istek B örneğine yönlendirilebilir.

Kayıt yalnızca A'nın belleğinde tutuluyorsa B bu bilgiye erişemez. Ayrıca A yeniden başlatıldığında kayıt kaybolur. Kayıt ortak bir kalıcı veri deposunda saklanıyorsa her iki örnek de gerekli bilgiye ulaşabilir. Böylece sorgunun doğruluğu, önceki isteği hangi örneğin karşıladığına bağlı kalmaz.

Bu tasarımda kayıt hizmetinin çalışma örnekleri durumsuz olabilir; veriyi koruyan veri hizmeti ise durum bilgilidir. *AWS Well-Architected Framework,* çalışma örneklerinin değiştirilebilmesini ve yatay ölçeklemeyi kolaylaştırmak için, istekler arasında gerekli durumun örneğin yerel belleğine veya diskine bağımlı olmamasını önerir. Korunması gereken durum, uygun harici veri hizmetlerine taşınabilir. [7]

Durum bilgili bileşenler

Veritabanı, kayıtların daha sonra da okunabilmesi için durumu korur. Benzer biçimde kalıcı mesaj saklayan bir mesaj aracısı, henüz tamamlanmamış işleri hatırlamak

zorundadır. Bu bileşenlerin yeniden başlatılması veya çoğaltılması sırasında verinin korunması ve örnekler arasındaki tutarlılık dikkate alınır.

Durum bilgili olmak yanlış bir tasarım değildir. Uygulamaların kayıtları ve işlem sonuçlarını koruması gerekir. Önemli olan, bu sorumluluğun uygun bileşene verilmesidir.

Bulutta çoğaltma ve yeniden başlatma

Hafta 6'da ele alınan Kubernetes, bir uygulama örneğini yeniden oluşturabilir veya aynı uygulamanın birden fazla örneğini çalıştırabilir. Ancak yeni örnek, önceki örneğin belleğindeki bilgileri kendiliğinden devralmaz. Bu nedenle gerekli iş durumunun tek bir örneğe bağlı tutulması, altyapının çoğaltma olanaklarından yararlanmayı zorlaştırır.

Durumsuz çalışma örneklerinde de geçici bilgi bulunabilir. Bir isteğin işlenmesi sırasında hesaplanan ara sonuçlar bellekte tutulabilir. Ancak gelecekteki bir isteğin doğruluğu, bu geçici bilginin hâlâ mevcut olmasına dayanmamalıdır. [7]

Etkinlik uygulamasında bunun pratik karşılığı şöyledir: Kayıt isteğini hangi örnek karşılarsa karşılasın, oluşturulan kayıt kalıcı olarak korunmalı ve sonraki sorgularla bulunabilmelidir. Çalışma örneğinin yaşam süresi ile iş verisinin yaşam süresi birbirinden ayrılmalıdır.

3.2. REST ve Temel API Tasarımı

Hizmet sözleşmesinde tanımlanan işlemlerin başka uygulamalar tarafından kullanılabilmesi için bir arayüz gerekir. *API — Application Programming Interface (uygulama programlama arayüzü)*, yazılım bileşenlerinin hangi işlemleri nasıl çağırabileceğini belirler. Bulut uygulamalarında bu arayüzler, web ve mobil istemcilerin hizmetlere erişmesini veya hizmetlerin birbirleriyle haberleşmesini sağlar.

REST — Representational State Transfer (temsili durum aktarımı), kaynaklar ve bu kaynakların temsilleri üzerinden etkileşimi düzenleyen bir mimari stildir. Bir programlama dili, ürün veya mesajlaşma protokolü değildir. REST yaklaşımına uygun web arayüzleri yaygın olarak *HTTP — Hypertext Transfer Protocol (hipermetin aktarım protokolü)* üzerinden gerçekleştirilir. [8]

Kaynak, adres ve temsil

Bir *resource (kaynak)*, uygulamanın erişime sunduğu anlamlı bir bilgi veya varlıktır. Etkinlik uygulamasında bir etkinlik, katılımcı kaydı veya etkinlik listesi kaynak olabilir.

Kaynaklar adreslerle tanımlanır. Örneğin:

- `/etkinlikler`: Etkinlik koleksiyonu.
- `/etkinlikler/42`: Kimliği 42 olan etkinlik.
- `/kayitlar/317`: Kimliği 317 olan katılımcı kaydı.

Kaynağa erişildiğinde istemciye kaynağın kendisi değil, onu açıklayan bir *representation (temsil)* gönderilir. Bu temsil, örneğin *JSON — JavaScript Object Notation (JavaScript nesne gösterimi)* biçiminde olabilir. JSON kullanılması tek başına arayüzü REST yapmaz; farklı temsil biçimleri de kullanılabilir.

HTTP yöntemleriyle işlem belirtme

Kaynak adresi, hangi bilgiye erişildiğini; HTTP yöntemi ise bu kaynak üzerinde istenen işlemin anlamını belirtir. Böylece aynı adres farklı yöntemlerle kullanılabilir. *Endpoint (uç nokta)*, arayüzde isteğin gönderildiği erişim noktasıdır; işlemi anlamak için adresle birlikte yöntem de dikkate alınır.

Tablo 4, etkinlik uygulamasındaki temel yöntem kullanımını göstermektedir. Yöntemlerin anlamları HTTP standardına dayanır. [9]

Tablo 4. Temel HTTP yöntemleri ve etkinlik uygulamasındaki örnek kullanımları.

Yöntem	Temel anlamı	Örnek kullanım
GET	Kaynağın temsilini almak	`GET /etkinlikler/42` ile etkinlik bilgilerini sorgulamak
POST	Gönderilen içeriği hedef kaynağın kurallarına göre işlemek	`POST /kayitlar` ile yeni kayıt oluşturmak
PUT	Hedef kaynağı verilen temsille oluşturmak veya mevcut temsilini değiştirmek	`PUT /etkinlikler/42` ile etkinliğin düzenlenebilir bilgilerinin tamamını göndermek
DELETE	Hedef kaynak ile mevcut işlevselliği arasındaki ilişkiyi kaldırmak	`DELETE /kayitlar/317` ile uygulamanın kurallarına göre kaydı kaldırmak

POST yalnızca kayıt oluşturmaya özgü değildir; örnekte bu amaçla kullanılmıştır. PUT ise yalnız değişen bir alanın gönderilmesiyle aynı anlama gelmez.

Kısa bir istek–yanıt örneği

Katılımcıyı etkinliğe kaydetmek için istemci aşağıdaki isteği gönderebilir. Gösterim, temel alanları içerecek şekilde sadeleştirilmiştir:

```
POST /kayitlar HTTP/1.1
Content-Type: application/json

{
  "etkinlikId": 42,
  "katilimciId": 108
}
```

Kayıt hizmeti gerekli kontrolleri yapıp kaydı oluşturduğunda şu yanıtı verebilir:

```
HTTP/1.1 201 Created
Location: /kayitlar/317
Content-Type: application/json
```

```
{
  "kayitId": 317,
  "etkinlikId": 42,
  "katilimciId": 108,
  "durum": "onaylandi"
}
```

Burada Content-Type, içeriğin biçimini belirtir. 201 Created, yeni kaynak oluşturulduğunu; Location ise oluşturulan kaynağın adresini bildirir. İstemci daha sonra GET /kayitlar/317 isteğiyle kaydı sorgulayabilir. 200 OK başarılı sorguda, 400 Bad Request hatalı girdide ve 404 Not Found bulunamayan kaynak için kullanılabilecek temel yanıt kodlarıdır. [9]

REST'in bulut tasarımıyla ilişkisi

REST'in durumsuz etkileşim kuralına göre her istek, anlaşılması için gerekli bilgileri taşımalıdır; sunucunun önceki istemci konuşmasını hatırlamasına dayanmamalıdır. Bu, kaynakların sunucuda kalıcı olarak saklanmasını engellemez. Önceki alt başlıkta ele alınan durumsuz çalışma örnekleriyle ilişkili olmakla birlikte, REST'in kuralı doğrudan istemci–sunucu etkileşimine yöneliktir. [8]

REST ayrıca istemci–sunucu ayrımı, önbelleklenebilirlik, tekbiçimli arayüz ve katmanlı sistem gibi kısıtlar içerir. Tekbiçimli arayüz, kaynakların tanımlanmasını, temsiller üzerinden işlem yapılmasını, mesajların anlaşılır olmasını ve bağlantılarla sonraki etkileşimlerin yönlendirilmesini kapsar. [8]

Buluttaki kayıt hizmetinin farklı örnekleri aynı arayüzü sunduğunda istemci, işlemi hangi örneğin yürüttüğünü bilmeden hizmeti kullanabilir. Ancak uygun adresler ve HTTP yöntemleri tasarlamak, kontenjan doğruluğu veya tekrar kayıtların önlenmesi gibi iş kurallarını kendiliğinden sağlamaz. Bu kurallar hizmet tarafından uygulanmalıdır.

3.3. API Sözleşmesi, Doğrulama ve İşlem Durumu

Bölüm 2.1'de hizmet sözleşmesinin, bir hizmetin kullanım kurallarını tanımladığı açıklanmıştır. API tasarımında bu kurallar; istek alanları, yanıtlar ve hata davranışlarıyla somutlaştırılır. İstemci, hangi bilgileri göndermesi gerektiğini ve aldığı yanıtın ne anlama geldiğini açıkça anlayabilmelidir.

Girdilerin ve iş kurallarının doğrulanması

Bir isteğin doğru biçimde yazılması, işlemin gerçekleştirilebileceği anlamına gelmez. Etkinlik kaydı oluşturulurken iki tür kontrol yapılır:

- **Girdi doğrulaması:** Zorunlu alanların bulunması, veri türlerinin ve değerlerin uygun olması kontrol edilir. Örneğin etkinlik kimliği eksik veya beklenen türde değilse istek reddedilir.

- **İş kuralı doğrulaması:** İsteğin uygulamanın kurallarına uygunluğu kontrol edilir. Etkinliğin kayıt kabul etmesi, kontenjan bulunması ve katılımcının daha önce kaydolmamış olması bu kapsamdadır.

Bu kontroller yalnızca kullanıcı arayüzüne bırakılmamalıdır. Aynı hizmete farklı istemciler erişebileceğinden, hizmet kendi kurallarını uygulamalıdır. Ayrıca kontenjan gibi eşzamanlı işlemlerden etkilenebilen kurallar, yalnızca ön kontrolle değil, kaydın oluşturulduğu veri işlemi sırasında da korunmalıdır.

Hata yanıtı, istemcinin sorunu anlamasına yardımcı olmalıdır. Örneğin uygulamanın sözleşmesinde kontenjan doluluğu, mevcut kaynak durumuyla çatışma olarak tanımlanabilir:

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "kod": "KONTENJAN_DOLU",
  "mesaj": "Etkinlik için boş kontenjan bulunmuyor."
}
```

`409 Conflict`, isteğin hedef kaynağın mevcut durumuyla çatışması nedeniyle tamamlanamadığını belirtir. Buradaki `kod` ve `mesaj` alanları ise örnek uygulamanın sözleşmesine aittir; bütün API'ler için zorunlu alan adları değildir. [9]

Kabul edilen iş ile tamamlanan iş

Bazı işlemler yanıt verilmeden tamamlanabilir. Önceki örnekte kayıt oluşturulmuş ve `201 Created` yanıtı verilmiştir. Ancak katılım belgesi hazırlama gibi uzun süren bir işte, istemciyi bütün işlem boyunca bekletmek yerine istek kabul edilip sonuç daha sonra sunulabilir.

`202 Accepted`, isteğin işlenmek üzere kabul edildiğini, ancak işlemenin henüz tamamlanmadığını belirtir. İşin daha sonra başarıyla tamamlanacağını garanti etmez. [9] Bu nedenle yanıt, istemciye işi nasıl takip edeceğini de açıklamalıdır. Ayrı bir durum sorgulama kaynağı kullanılması, bu amaçla uygulanabilen bir tasarım örüntüsüdür. [10]

Örneğin `POST /belge-isleri` isteğine şu yanıt verilebilir:

```
HTTP/1.1 202 Accepted
Location: /islemler/is-824
Content-Type: application/json

{
  "islemId": "is-824",
  "durum": "bekliyor"
}
```

İstemci daha sonra `GET /islemler/is-824` isteğiyle işin durumunu sorgular. Durum bilgisi; `bekliyor`, `isleniyor`, `tamamlandi` veya `basarisiz` gibi sözleşmede açıklanan değerlerle

gösterilebilir. Tamamlanan işte sonuç adresi, başarısız işte ise anlaşılır hata bilgisi sunulur.

Burada önemli bir ayrım vardır: Durum sorgusunun `200 OK` yanıtı vermesi, arka plan işinin başarıyla tamamlandığını göstermez. Bu kod sorgunun başarılı olduğunu belirtir; işin sonucu yanıt içindeki durum alanından anlaşılır. [10]

Bulutta işlem durumunun korunması

Durum sorgusu, ilk isteği kabul eden örnekten farklı bir uygulama örneğine ulaşabilir. Bu nedenle iş kimliği ve durum bilgisi, yalnızca ilk örneğin belleğinde tutulmamalıdır. Örneğin belge işçisi yeniden başlatıldığında istemci hâlâ aynı işlem kimliğiyle işini takip edebilmelidir.

İstemci yanıtı alamadığında aynı isteği tekrar gönderebilir. API sözleşmesi, bu tekrarın yeni bir iş oluşturup oluşturmayacağını da belirlemelidir. Aynı mantıksal işin tanınması ve ek iş etkisinin önlenmesi, mesajlaşma bölümünde ele alınacaktır.

3.4. API Ağ Geçidi, Hizmet Keşfi ve Yük Dengeleme

Bir hizmetin arayüzünü tanımlamak, ona nasıl ulaşılacağını tek başına çözmez. Bulut uygulamalarında farklı hizmetler ve aynı hizmetin birden fazla çalışma örneği bulunabilir. Örnekler yeniden oluşturulduğunda adresleri de değişebilir. Bu nedenle "isteğin doğru hizmete yönlendirilmesi, hizmetin bulunması ve uygun örneğe iletilmesi" ayrı görevler olarak ele alınır.

API ağ geçidi

API gateway (API ağ geçidi), istemcilerin uygulama hizmetlerine eriştiği ortak giriş katmanıdır. İstekleri uygun hizmetlere yönlendirir; yapılandırmasına göre kimlik doğrulama, istek hızını sınırlama ve ortak erişim politikalarını uygulama gibi görevler de üstlenebilir. Böylece istemcinin her arka uç hizmetinin adresini ayrı ayrı bilmesi gerekmez. [11]

Etkinlik uygulamasında istemci, `/etkinlikler` ve `/kayitlar` adreslerine aynı giriş noktası üzerinden erişebilir. Ağ geçidi etkinlik sorgusunu etkinlik hizmetine, kayıt isteğini kayıt hizmetine iletir.

Ancak kontenjan kontrolü veya katılımcının kayıt koşullarının değerlendirilmesi gibi iş kuralları kayıt hizmetinde kalmalıdır. Ağ geçidinin görevi, bütün uygulamanın iş mantığını merkezileştirmek değildir. Ayrıca ağ geçidi ek bir bileşen olduğundan, erişilebilirliği ve kapasitesi de uygulamanın tasarımında dikkate alınmalıdır.

Hizmet keşfi

Service discovery (hizmet keşfi), bir hizmete ulaşmak için kullanılabilecek güncel erişim noktalarının bulunmasıdır. Hizmet adreslerini uygulama koduna sabitlemek, çalışma

örnekleri değiştiğinde iletişimin kesilmesine neden olabilir. Bunun yerine mantıksal hizmet adları ve platformun sağladığı keşif mekanizmaları kullanılır.

Hafta 6'da ele alınan Kubernetes Service nesnesi bu ihtiyaca örnektir. Service, değişebilen Pod kümesine erişmek için kararlı bir soyutlama sağlar. DNS — Domain Name System (alan adı sistemi) üzerinden hizmet adı çözümlenebilir; normal bir Service yapılandırmasında trafik, uygun arka uç Pod'lara iletilir. Pod'lar değişse bile çağıran bileşenin her Pod adresini takip etmesi gerekmez. [12]

Örneğin kayıt hizmeti, bildirim hizmetinin belirli bir çalışma örneğinin adresini bilmek yerine `bildirim-hizmeti` gibi mantıksal bir ad kullanabilir. Platforma göre bu ad kararlı bir hizmet adresine veya erişilebilir örneklerin adreslerine çözümlenebilir.

Yük dengeleme

Load balancing (yük dengeleme), trafiğin aynı hizmetin uygun çalışma örnekleri arasında dağıtılmasıdır. Hafta 3'te ağ açısından incelenen bu işlev, burada hizmetlerin çoğaltılmasıyla ilişkilidir.

Kayıt hizmetinin A ve B örnekleri bulunduğunda yük dengeleme, gelen isteği bu örneklerden birine iletir. Hizmet keşfi erişim noktalarını bulmaya, yük dengeleme ise trafik dağıtımına yardımcı olur. API ağ geçidi de önce isteğin hangi hizmete ait olduğunu belirleyebilir. Tablo 5, bu görevlerin ayrımını özetlemektedir.

Tablo 5. API ağ geçidi, hizmet keşfi ve yük dengelemenin temel görevleri.

Bileşen veya Mekanizma	Temel Soru	Etkinlik Uygulamasındaki Örnek
API ağ geçidi	İstek hangi hizmete yönlendirilmeli?	`/kayitlar` isteğini kayıt hizmetine iletmek
Hizmet keşfi	Hizmete hangi erişim noktasından ulaşılabilir?	Kayıt hizmetinin mantıksal adını çözümlemek
Yük dengeleme	İsteği hangi çalışma örneği karşılamalı?	İsteği kayıt hizmetinin A veya B örneğine iletmek

Bu görevler farklı ürünlerde birlikte sunulabilir. Her görev için ayrı bir cihaz veya yazılım kurulması zorunlu değildir.

Hizmetler arası iletişimin yönetimi

Service mesh (hizmet ağı), hizmetler arasındaki iletişime ortak trafik, güvenlik ve gözlem politikaları uygulanmasını sağlayan altyapı yaklaşımıdır. API ağ geçidinin istemci erişimindeki rolünden farklı olarak, hizmet ağı ağırlıklı biçimde uygulamanın iç iletişimini düzenler; dış erişim bileşenleriyle de bütünleşebilir. [13]

Hizmet ağı kullanımı zorunlu değildir; sağlayacağı yarar, hizmetler arası iletişimin karmaşıklığı ve ek yönetim yükü birlikte değerlendirilerek kararlaştırılır.

4. Hizmetler Arası İletişim, Mesajlaşma ve Olay Güdümlü Mimari

Bir bulut uygulamasında hizmet sınırlarının ve erişim noktalarının belirlenmesi kadar, hizmetlerin ne zaman ve nasıl haberleşeceği de önemlidir. Bazı işlemlerde çağıran bileşenin sonucu hemen alması gerekirken, bazı işler daha sonra tamamlanabilir. Bu farklılık, iletişim yönteminin seçimini etkiler.

Örneğin etkinlik bilgilerinin sorgulanmasında istemci yanıtı bekler. Kayıt sonrasında bildirim gönderilmesi ise ayrı bir iş olarak yürütülebilir. Hizmetlerin doğrudan çağrılar, kuyruklar veya olaylar üzerinden iletişim kurması; yanıt süresini, yükün karşılanmasını ve bir bileşendeki gecikmenin diğerlerine etkisini değiştirir. Bu bölümde iletişim modelleri bu gereksinimler üzerinden incelenecek; ağ hataları ve tekrarlanan mesajlar karşısında işlem doğruluğunun nasıl korunacağı açıklanacaktır.

4.1. Eşzamanlı ve Eşzamansız İletişim

Hizmetler arasındaki iletişimde temel seçimlerden biri, çağıran bileşenin iş akışını sürdürmek için karşı tarafın sonucunu bekleyip beklemeyeceğidir. *Synchronous communication (eşzamanlı iletişim)* ve *asynchronous communication (eşzamansız iletişim)* bu bağımlılık bakımından farklılaşır. Buradaki "eşzamanlı" ifadesi, iki hizmetin aynı anda çalışmasından çok, çağıranın yanıtı beklemesini ifade eder. [14]

Eşzamanlı iletişim

Eşzamanlı iletişimde bir bileşen başka bir hizmete istek gönderir ve ilgili işlemi sürdürebilmek için yanıtı bekler. Örneğin etkinlik ekranını hazırlayan uygulama, etkinlik hizmetinden adı, zamanı ve yeri sorgular. Ekranın doğru gösterilebilmesi için bu bilgilere ihtiyaç vardır.

Bu yaklaşımda istek ve sonuç arasındaki ilişki açıktır. Ancak çağrılan hizmetin yanıt süresi, çağıranın yanıt süresini de etkiler. Bir işlem art arda birkaç hizmetin yanıtına ihtiyaç duyuyorsa, gecikmeler birikebilir. Bağımlı hizmetin erişilememesi de işlemin tamamlanmasını engelleyebilir. [14]

Örneğin kayıt hizmeti, kullanıcıya yanıt vermeden önce bildirim hizmetinin yanıtını bekliyorsa, bildirim sağlayıcısındaki yavaşlık kayıt yanıtını da geciktirir. Bildirimin tamamlanması kayıt başarısı için zorunlu değilse bu bağımlılık yeniden değerlendirilmelidir.

Eşzamansız iletişim

Eşzamansız iletişimde işin iletilmesi ile tamamlanması birbirinden ayrılır. Kayıt hizmeti, bildirim işini bir kuyruğa aktarabilir ve bildirimin gönderilmesini beklemeden kayıt sonucunu döndürebilir. Bildirim hizmeti işi daha sonra alıp gerçekleştirir.

Gönderici, mesajın aracı tarafından kabul edildiğine ilişkin onayı bekleyebilir. Bu onay, alıcının işi tamamladığı anlamına gelmez. Bölüm 3.3'te açıklanan kabul edilen iş ile tamamlanan iş ayrımı burada da geçerlidir.

Kuyruk, ani yük artışlarında tampon görevi görebilir. Örneğin kayıtların kısa sürede yoğunlaşması durumunda bildirim işleri kuyrukta birikir; tüketici bunları kendi kapasitesine göre işler. Ancak tüketim kapasitesi sürekli yetersiz kalırsa bekleme süresi büyür. Kuyruk yükü ortadan kaldırmaz, işin zamana yayılmasını sağlar. [15]

Eşzamansız iletişim, işin durumunu takip etme ve başarısız işleri yönetme ihtiyacını da beraberinde getirir. Kullanıcının "kaydım tamamlandı" ile "bildirimim gönderildi" durumlarını ayrı görebilmesi gerekebilir.

İletişim yönteminin seçimi

Tablo 6, iki yaklaşımın etkinlik uygulamasındaki kullanımını karşılaştırmaktadır.

Tablo 6. Eşzamanlı ve eşzamansız iletişimin karşılaştırılması.

Ölçüt	Eşzamanlı İletişim	Eşzamansız İletişim
Sonuca bağımlılık	Akış, karşı tarafın yanıtını bekler	İşin iletilmesi ve tamamlanması ayrılır
Örnek	Etkinlik bilgilerini sorgulama	Kayıt sonrası bildirim gönderme
Gecikmenin etkisi	Çağıranın bekleme süresine yansır	İşin tamamlanması gecikebilir
Sonucun takibi	Genellikle doğrudan yanıttan anlaşılır	Ayrı durum kaydı veya sonuç bildirimi gerekebilir

Aynı bulut uygulamasında iki yöntem birlikte kullanılabilir. Seçim, kullanılan teknolojiden önce sonucun ne zaman gerekli olduğuna ve gecikmenin iş açısından kabul edilip edilemeyeceğine dayanmalıdır.

Kodda eşzamansız çalışma ile mimari ayrım

Bir HTTP çağrısında `async/await` kullanılması, uygulamanın yanıt gelene kadar diğer işleri yürütmesine olanak tanır. Ancak yanıtı bekleyen işlemin sonraki adımı hâlâ bu sonuca bağlı olabilir. Örneğin kayıt hizmeti, bildirim hizmetini `await` ile çağırıyor ve kullanıcıya yanıt vermeden önce gönderim sonucunu bekliyorsa, diğer kayıt istekleri işlenebilse bile bu kullanıcının kayıt yanıtı bildirim tamamlanana kadar gecikir. Kod eşzamansız yazılmış olsa da hizmetler arasındaki sonuç bekleme bağımlılığı sürer. [14]

Buna karşılık kayıt hizmeti bildirim işini kuyruğa aktarır ve mesajın kabul edildiğini öğrendikten sonra bildirimin gönderilmesini beklemeden kullanıcıya kayıt sonucunu döndürürse, iki işin tamamlanması birbirinden ayrılmış olur. Dolayısıyla `async/await`, kodun beklemeyi nasıl yönettiğiyle; eşzamansız mesajlaşma ise hizmetler arasındaki iş akışının nasıl düzenlendiğiyle ilgilidir.

4.2. İş Kuyruğu ve Yayınla–Abone Ol Modeli

Eşzamansız iletişimde hizmetler arasında taşınan veri, bir *message (mesaj)* olarak düzenlenebilir. Mesaj, yapılması istenen bir işi veya gerçekleşmiş bir durumu bildirebilir. Mesajlaşma sisteminde üç temel rol bulunur:

- **Producer (üretici):** Mesajı oluşturan ve gönderen bileşendir.
- **Message broker (mesaj aracısı):** Mesajların ilgili alıcılara aktarılmasını düzenleyen bileşendir.
- **Consumer (tüketici):** Mesajı alan ve işleyen bileşendir.

Mesajın hangi tüketicilere iletileceği, kullanılan iletişim modeline bağlıdır. İş kuyruğu ile yayınla–abone ol modeli arasındaki temel fark, bir işin paylaşılması ile aynı bilginin bağımsız ilgililere dağıtılmasıdır.

İş kuyruğu

Queue (kuyruk), mesajların tüketiciler tarafından işlenmek üzere bekletildiği yapıdır. İş kuyruğunda birden fazla tüketici, aynı tür işleri paylaşarak gerçekleştirir. Normal işleyişte her mesaj, tüketicilerden biri tarafından işlenmek üzere alınır. Bu düzen, *competing consumers (iş paylaşan tüketiciler)* yaklaşımı olarak da adlandırılır. [16]

Örneğin bildirim kuyruğunda 100 gönderim işi bulunsun. Bildirim hizmetinin iki çalışma örneği, farklı işleri alarak aynı anda çalışabilir. Amaç, her bildirimi iki örneğin de göndermesi değil, bekleyen işleri paylaşmalarıdır. İşlerin süresi değişebileceğinden her örneğin tam olarak 50 iş alması gerekmez.

Tüketici işi tamamladığında mesajlaşma sistemine tamamlanma onayı verebilir. Başarısızlık veya onayın kaybolması durumunda mesaj yeniden teslim edilebilir. Dolayısıyla iş kuyruğu kullanılması, bir işin hiçbir koşulda tekrar işlenmeyeceği anlamına gelmez. [16]

Yayınla–abone ol modeli

Publish–subscribe (yayınla–abone ol) modelinde üretici bir mesaj yayımlar; bu mesaj, ilgilenen bağımsız aboneliklere dağıtılır. Üreticinin bütün alıcıları ayrı ayrı çağırması gerekmez. Abonelikler, yapılandırmalarına göre ilgili mesajları alır. [17]

Örneğin “Katılımcı kaydı oluşturuldu” bilgisi hem bildirim hem raporlama hizmetini ilgilendirebilir:

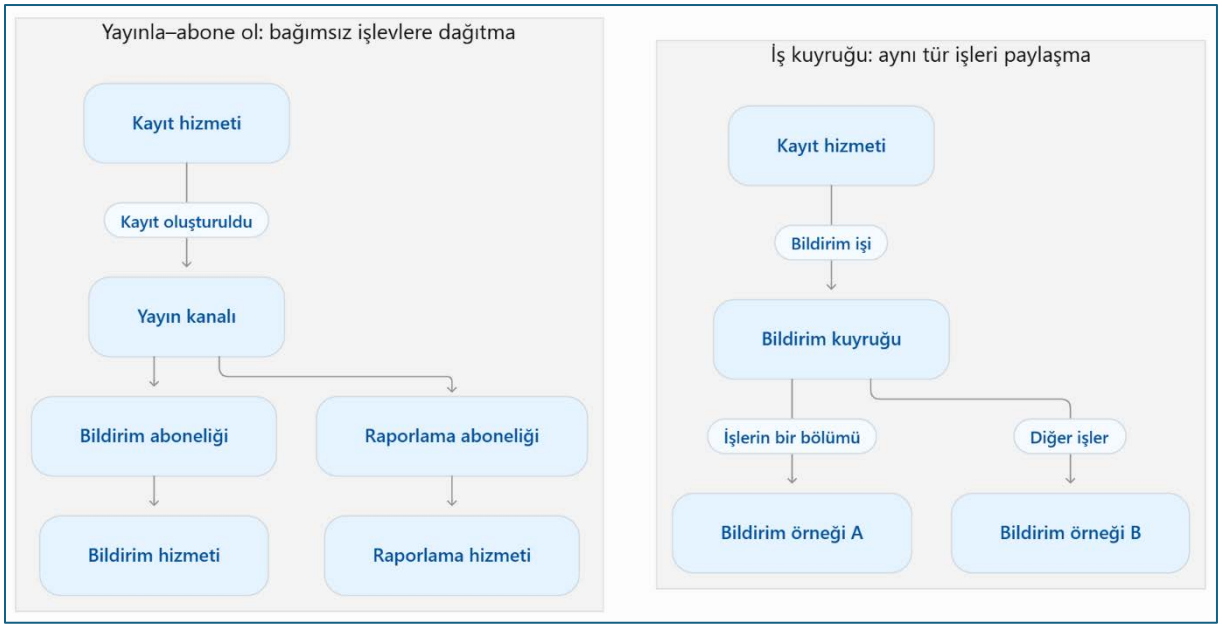
- Bildirim aboneliği, katılımcıya mesaj gönderilmesini sağlar.
- Raporlama aboneliği, kayıt istatistiklerini günceller.

Bu iki işlevin de aynı kayıt bilgisini alması gerekir. İkisi tek bir iş kuyruğunu paylaşırsa mesaj yalnızca birine teslim edilebilir; diğer işlevin gerçekleşmesi sağlanamaz. Bağımsız abonelikler bu ihtiyacı karşılar.

Bir aboneliğin kendi içinde birden fazla tüketicisi de bulunabilir. Örneğin bildirim aboneliğine bağlı iki örnek bildirim işlerini paylaşırken, raporlama aboneliği aynı yayımlanmış bilgiyi ayrıca alabilir.

Modellerin karşılaştırılması

Şekil 2'de üst yapı bir bildirim işinin tüketiciler arasında paylaşılmasını, alt yapı ise yayımlanan kayıt bilgisinin farklı aboneliklere dağıtılmasını göstermektedir. Kuyruktan çıkan iki ok, her mesajın iki tüketiciye birden gönderildiğini göstermez.



Şekil 2. İş kuyruğunda yük paylaşımı ve yayınla–abone ol modelinde bağımsız dağıtım.

Buluttaki kullanım ve kalıcılık

İş kuyruğu, aynı görevi yapan çalışma örneklerinin çoğaltılmasını destekler. Yayınla–abone ol modeli ise mevcut üreticinin her yeni hizmeti doğrudan çağırmasına gerek kalmadan yeni işlevlerin eklenmesini kolaylaştırır.

İki modelde de mesajların saklanma süresi ve tüketici kapalıyken korunup korunmadığı açıkça belirlenmelidir. Kuyruk veya abonelik bulunması tek başına kalıcılık garantisi değildir. Teslim davranışı, mesajlaşma hizmetinin özelliklerine ve yapılandırmaya bağlıdır.

4.3. Olay, Komut ve Olay Güdümlü Akış

Mesajlaşmada yalnızca mesajın nasıl taşındığı değil, ne anlama geldiği de önemlidir. Bir mesaj, bir işlemin yapılmasını isteyebilir veya gerçekleşmiş bir durumu bildirebilir. Bu ayrım, hizmetlerin sorumluluklarını ve birbirlerinden beklentilerini belirler.

Event (olay), sistemde gerçekleşen bir durumu veya değişikliği bildirir. *Command (komut)* ise bir işlemin gerçekleştirilmesini ister. Örneğin "Katılımcı kaydı oluşturuldu" bir olaydır; "Katılımcıya bildirim gönder" bir komuttur.

Tablo 7, aynı uygulamadaki olay ve komut örneklerini göstermektedir.

Tablo 7. Olay ve komut mesajlarının anlam bakımından karşılaştırılması.

Özellik	Olay	Komut
Temel anlam	Bir durum gerçekleşti	Bir işlem gerçekleştirilsin
Örnek	KatilimciKaydiOlusturuldu	BildirimGonder
Gönderenin amacı	Gerçekleşen durumu duyurmak	Belirli bir işi yaptırmak
Alıcının rolü	İlgiliyse kendi sorumluluğuna göre tepki vermek	İstenen işlemi değerlendirip gerçekleştirmek

Bir komutun gönderilmiş olması, işlemin başarıyla gerçekleştiğini göstermez. Örneğin KayitOlustur komutu kontenjan doluluğu nedeniyle reddedilebilir. KatilimciKaydiOlusturuldu olayı ise kayıt gerçekten oluşturulduktan sonra üretilmelidir.

Olay güdümlü mimari

Event-driven architecture (olay güdümlü mimari), bileşenlerin gerçekleşen olaylara tepki vererek çalıştığı yaklaşımdır. Olay üreticileri değişiklikleri bildirir, olay kanalları bunları taşır ve tüketiciler ilgili işlemleri gerçekleştirir. Üretici, olayı işleyecek bütün tüketicileri ayrı ayrı çağırmak zorunda değildir. [18]

Etkinlik uygulamasında örnek akış şöyle olabilir:

1. Kayıt hizmeti, katılımcı kaydını oluşturur ve kalıcı olarak saklar.
2. KatilimciKaydiOlusturuldu olayını yayımlar.
3. Bildirim hizmeti olayı alarak gönderim işini başlatır.
4. Raporlama hizmeti aynı olayı kendi aboneliğinden alarak istatistikleri günceller.

Kayıt hizmetinin temel sorumluluğu kayıt oluşturmaktır. Bildirim ve raporlama hizmetleri, bu olay karşısında kendi sorumluluklarını yerine getirir. Daha sonra farklı bir tüketici eklenirse, kayıt hizmetinin onu doğrudan çağıracak biçimde değiştirilmesi gerekmeyebilir.

Olayın içeriği ve sözleşmesi

Olayın alıcı tarafından anlaşılabilmesi için türü, kimliği ve taşıdığı verilerin anlamı açık olmalıdır. CloudEvents belirtimi de olayların ortak biçimde tanımlanabilmesi için olay kimliği, kaynağı ve türü gibi üst verileri düzenler. [19]

Aşağıdaki JSON, ders uygulaması için hazırlanmış basitleştirilmiş bir olay örneğidir; CloudEvents biçiminin doğrudan uygulaması değildir:

```
{
  "olayId": "olay-8421",
  "olayTuru": "KatilimciKaydiOlusturuldu",
  "semaSurumu": 1,
  "gerceklesmeZamani": "2026-10-14T08:00:00Z",
  "veri": {
    "kayitId": 317,
    "etkinlikId": 42,
    "katilimciId": 108
  }
}
```

olayId, bu olayın kimliğidir; etkinlikId ise kayıt yapılan etkinliği belirtir. Bunlar farklı amaçlara hizmet eder. semaSurumu, mesaj yapısının hangi sürümde olduğunu gösterir.

Örnekte yalnızca kimlikler taşınmıştır. Bildirim hizmeti ek bilgiye ihtiyaç duyarsa ilgili hizmetten sorgulayabilir. Alternatif olarak gerekli bilgilerin bir bölümü olaya eklenebilir. İlk seçenek ek çağrı gerektirirken, ikinci seçenek mesaj boyutunu ve paylaşılan veri miktarını artırır. Olay içeriği, tüketicilerin ihtiyacına göre belirlenmeli; bütün veritabanı kaydı gereksiz yere aktarılmamalıdır.

Buluttaki kullanım amacı

Olay güdümlü akış, farklı işlevlerin birbirini beklemeden ilerlemesini ve kendi yüklerine göre çalıştırılmasını destekler. Ancak bir olayın yayımlanması, bütün tüketicilerin işlerini tamamladığı anlamına gelmez. Bildirim gönderilmişken raporlama güncellemesi hâlâ bekliyor olabilir.

Bu nedenle her işlem olay güdümlü olmak zorunda değildir. Kullanıcının hemen sonuç beklediği sorgularda doğrudan istek–yanıt iletişimi kullanılabilir; aynı uygulamanın arka plan işleri olaylarla başlatılabilir. Tasarım, işin sonuç ve zamanlama gereksinimlerine göre yapılır. [18]

4.4. Kısmi Hata, Tekrar Teslim ve İdempotentlik

Dağıtık bir bulut uygulamasında bazı bileşenler çalışırken diğerleri erişilemez veya gecikmiş olabilir. *Partial failure (kısmi hata)*, sistemin bütünü durmadan belirli bir bileşenin ya da iletişimin başarısız olmasıdır. Örneğin kayıt hizmeti çalışırken bildirim hizmetine yapılan çağrı yanıt vermeyebilir.

Bu durumda çağıran bileşen her zaman işlemin sonucunu bilemez. İstek karşı tarafa ulaşmamış olabilir; işlem tamamlanmış fakat yanıt kaybolmuş da olabilir. Yanıt alınamaması, işlemin gerçekleşmediğini kesin olarak göstermez.

Zaman aşımı ve sınırlı yeniden deneme

Timeout (zaman aşımı), bir yanıt için beklenecek süreyi sınırlandırır. Süre dolduğunda çağıran bileşen beklemeyi bırakabilir; ancak bu, karşı tarafta başlayan işlemin otomatik olarak durduğu anlamına gelmez.

Retry (yeniden deneme), geçici hatalarda işlemin tekrar çağrılmasıdır. Deneme sayısı ve denemeler arasındaki bekleme sınırlandırılmalıdır. Sürekli ve hızlı tekrarlar, zaten yoğun olan hizmetin yükünü artırabilir. Geçersiz girdi gibi tekrarlandığında düzelmeyecek hatalar için yeniden deneme uygun değildir. [20]

Örneğin kullanıcı kayıt yanıtını alamayıp aynı isteği yeniden gönderirse, ilk kayıt oluşturulmuş olabilir. İkinci isteğin yeni bir kayıt oluşturması, ağdaki geçici bir sorunu veri doğruluğu sorununa dönüştürür.

Mesajın yeniden teslim edilmesi

Benzer belirsizlik mesajlaşmada da oluşabilir. Bir işçi mesajı alır, işi tamamlar; fakat tamamlanma onayını iletmeden kapanırsa mesaj yeniden teslim edilebilir. Bazı mesajlaşma hizmetleri, kendi teslim davranışları nedeniyle de aynı mesajı birden fazla kez iletebilir. Bu nedenle tüketici, tekrar teslim olasılığına uygun tasarlanmalıdır. [21]

Katılım belgesi oluşturma örneğinde:

1. İşçi, kayıt 317 için belge oluşturma mesajını alır.
2. Belgeyi oluşturur ve sonucu kalıcı olarak kaydeder.
3. Tamamlanma onayı iletilemeden işçi kapanır.
4. Mesaj başka bir işçiye yeniden teslim edilir.

İkinci işçi, mevcut sonucu tanımadan aynı işi yaparsa ikinci bir belge kaydı oluşturabilir.

İdempotent işlem

Idempotency (idempotentlik), aynı mantıksal işlemin tekrar uygulanmasının ilk başarılı uygulamadan sonra ek iş etkisi oluşturmamasıdır. Bu, işlem kodunun mutlaka yalnızca bir kez çalışması veya her çağrının aynı yanıtı vermesi demek değildir.

Belge örneğinde işin kimliği, kayıt kimliği ile belge türünden oluşturulabilir. İşçi, bu kimliğe ait tamamlanmış sonuç varsa mevcut sonucu kullanır; ikinci bir belge kaydı oluşturmaz. Böylece mesaj yeniden teslim edilse bile hedeflenen iş sonucu korunur.

Bu koruma yalnızca bellekte tutulan "işlenen işler" listesine dayanmamalıdır. İşçi yeniden başladığında liste kaybolabilir; başka bir işçi de aynı listeye erişemeyebilir. Kalıcı iş veya sonuç kaydı gerekir.

Ayrıca "önce kontrol et, sonra oluştur" yaklaşımı tek başına yeterli olmayabilir. İki işçi aynı anda kontrolü geçebilir. Benzersizlik kısıtı ve uygun veri işlemi, aynı iş için iki sonucun oluşmasını önleyecek şekilde birlikte kullanılmalıdır. Harici bir sisteme gönderim gibi yan etkilerde ise yalnızca yerel veritabanı kaydı yeterli olmayabilir; karşı hizmetin tekrar önleme desteği de değerlendirilir.

Teslim güvencesi ve işlem doğruluğu

Bir mesajlaşma hizmetinin *at-least-once (en az bir kez)* teslim yaklaşımı, tekrar teslim olabileceğini ifade eder. *Exactly-once (tam bir kez)* olarak sunulan bir özellikte ise güvencenin hangi işlemleri ve koşulları kapsadığı incelenmelidir. Mesajın teslimine ilişkin güvence, bütün dış sistemlerdeki iş etkilerinin kendiliğinden yalnızca bir kez gerçekleşmesini sağlamaz.

Dolayısıyla yeniden deneme ve idempotentlik birlikte düşünülmelidir: Yeniden deneme, geçici hatadan sonra işi sürdürebilmeyi; idempotent tasarım ise bu tekrarın kayıt, belge veya ödeme gibi iş sonuçlarını çoğaltmamasını amaçlar. [20]

4.5. Nihai Tutarlılık ve Başarısız İşlerin Yönetimi

Olaylarla haberleşen hizmetler, aynı değişikliği farklı zamanlarda işleyebilir. Kayıt hizmetinde tamamlanan bir işlem, bildirim veya raporlama hizmetine henüz yansımamış olabilir. Bu nedenle dağıtık uygulamada "işin tamamlanması, değişikliğin diğer hizmetlere aktarılması ve bu hizmetlerce işlenmesi" birbirinden ayrılmalıdır.

Nihai tutarlılık

Eventual consistency (nihai tutarlılık), yeni güncellemeler olmadığında ve gerekli değişiklikler başarıyla aktarılıp işlendiğinde, ilgili veri görünümlerinin zamanla tutarlı hâle gelmesidir. Bütün hizmetlerin aynı bilgiyi aynı anda göstermesi beklenmez. [18]

Örneğin katılımcının kaydı tamamlanmışken raporlama ekranındaki toplam kayıt sayısı kısa süre daha eski değeri gösterebilir. Raporlama hizmeti kayıt olayını işlediğinde sayı güncellenir. Böyle bir gecikme istatistik ekranında kabul edilebilir; ancak son kontenjanın iki kişiye verilmemesi gibi bir kural, gecikmeli rapor verisine dayandırılmamalıdır.

Nihai tutarlılık, "veri sonunda mutlaka düzelir" varsayımı değildir. Olay kaybolmuş veya tüketici sürekli başarısız olmuşsa gerekli güncelleme gerçekleşmeyebilir. Bu nedenle kabul edilebilir gecikmenin yanında, başarısız aktarım ve işleme durumlarının nasıl giderileceği de belirlenmelidir.

Başarısız mesajların ayrılması

Geçici bir bağlantı hatasında sınırlı yeniden deneme yararlı olabilir. Ancak zorunlu alanı eksik bir mesajı sürekli tekrar işlemek sorunu çözmez. Böyle bir mesaj, tüketim kapasitesini gereksiz yere kullanabilir.

Dead-letter queue — DLQ (başarısız mesaj kuyruğu), normal akışta işlenemeyen mesajların inceleme amacıyla ayrıldığı yapıdır. Mesaj, belirlenen teslim denemesi sınırı aşıldığında veya işlenemeyeceği tespit edildiğinde bu alana aktarılabilir. Aktarım koşulları kullanılan hizmete ve yapılandırmaya bağlıdır. [22]

Mesajın DLQ'ya taşınması, işin tamamlandığı veya hatanın düzeltildiği anlamına gelmez. İnceleme sırasında:

- Hatanın nedeni belirlenir.
- Veri veya uygulama sorunu giderilir.
- Uygunsa mesaj yeniden işleme alınır.
- Tekrar işlemenin ek iş etkisi oluşturması önlenir.

Örneğin belge oluşturma mesajında kayıt kimliği eksikse, aynı mesajı tekrar teslim etmek yeterli değildir. Mesajın neden hatalı üretildiği de araştırılmalıdır.

Veri kaydı ile olay yayınlama arasındaki boşluk

Başarısızlık yalnızca tüketicide oluşmaz. Kayıt hizmetinin önce katılımcı kaydını veritabanına yazdığını, ardından kayıt olayını yayımladığını düşünelim. Hizmet bu iki adım arasında kapanırsa kayıt oluşur; fakat bildirim ve raporlama hizmetleri değişiklikten haberdar olmaz.

Olayı önce yayımlamak da sorunu çözmez. Bu kez veri kaydı başarısız olursa tüketiciler, gerçekte oluşmamış bir kayda tepki verebilir. İki ayrı sisteme yapılan işlemler, kendiliğinden tek bir işlem güvencesine sahip değildir. [23]

Transactional outbox (işlemsel çıkış kutusu) yaklaşımında iş verisi ile yayımlanacak olay kaydı aynı yerel veritabanı işlemi içinde saklanır. Ayrı bir aktarım bileşeni, tamamlanmış işlemlere ait olay kayıtlarını okuyarak mesajlaşma sistemine gönderir. Böylece hizmet olay yayınlamadan kapanırsa, gönderilmesi gereken olay kalıcı kayıttan daha sonra bulunabilir. [23]

Bu yaklaşım bütün hizmetleri tek bir dağıtık işlem hâline getirmez. Olayın aktarımı ve tüketicilerin işlemesi daha sonra gerçekleşir. Aktarım sırasında tekrar gönderim de oluşabileceğinden, önceki bölümde açıklanan idempotentlik ihtiyacı devam eder.

Bulut uygulamasında bu mekanizmaların ortak amacı, geciken iş ile kaybolan veya başarısız işin ayırt edilebilmesini sağlamaktır. Kayıt tamamlandı, bildirim bekliyor ve belge oluşturma başarısız gibi durumların açık tutulması, kullanıcıya doğru bilgi verilmesini ve gerekli işlerin güvenli biçimde sürdürülmesini destekler.

5. Serverless ve Fonksiyon Tabanlı Çalıştırma

Önceki bölümlerde uygulamanın hizmetlere ayrılması ve bu hizmetlerin istekler veya mesajlar üzerinden haberleşmesi ele alınmıştır. Bu bölümde, belirli bir işi gerçekleştiren kodun bulutta nasıl çalıştırılacağına odaklanılmaktadır.

Serverless yaklaşımda sunucuların sağlanması ve çalışma ortamının yönetilmesi büyük ölçüde bulut platformuna bırakılır. Fonksiyon tabanlı çalıştırma, bu yaklaşımın bir

biçimidir: Bir istek veya olay, belirli bir işlevi gerçekleştiren kodu tetikler. Örneğin bir dosyanın yüklenmesi, içeriğini doğrulayan fonksiyonun çalışmasını başlatabilir. Bu modelin uygunluğu; işin süresi, kaynak ihtiyacı, durum yönetimi ve diğer hizmetlere bağımlılıkları üzerinden değerlendirilir.

5.1. Serverless Yaklaşım ve FaaS

Serverless yaklaşım, uygulamanın çalışması için gerekli sunucuların sağlanması ve altyapının yönetilmesi gibi görevleri büyük ölçüde bulut platformuna bırakır. Sunucular fiziksel olarak var olmaya devam eder; geliştiricinin bunları tek tek kurması ve işletmesi gereksinimi azalır. Böylece geliştirme çalışmasının ağırlığı, altyapı işletiminden uygulama davranışına kayar.

Serverless yalnızca fonksiyon çalıştırmakla sınırlı değildir. Konteyner tabanlı uygulamalar da sunucu veya küme yönetimi kullanıcıya bırakılmadan çalıştırılabilir. Örneğin Google Cloud Run, kod ve konteynerleri yönetilen altyapıda yürütür. Dolayısıyla konteyner kullanımı ile serverless yaklaşım birbirini dışlamaz. [24]

FaaS'ın serverless içindeki yeri

FaaS — Function as a Service (hizmet olarak fonksiyon), belirli bir işlevi gerçekleştiren kodun yönetilen çalışma ortamında, bir istek veya tetikleyiciyle çağrıldığı modeldir. Geliştirici fonksiyon kodunu ve gerekli yapılandırmayı sağlar; platform çağrının yürütüleceği ortamı yönetir.

Fonksiyon, örneğin bir HTTP isteğiyle, kuyruğa mesaj gelmesiyle veya depolamaya dosya yüklenmesiyle çalıştırılabilir. Azure Functions bu tür tetikleyicilerle kod çalıştırmayı destekler; AWS Lambda da sunucu sağlama ve yönetme gerektirmeden fonksiyon kodunu yürütür. [25], [26]

Buradaki "fonksiyon", yalnızca programlama dilindeki herhangi bir metodu ifade etmez. Platform tarafından çağrılabilen, girdiyi alıp belirli bir işi gerçekleştiren "dağıtılmış bir uygulama birimidir". İçinde birden fazla metot, kütüphane ve iş kuralı bulunabilir.

Küçük bir kullanım örneği

Etkinlik uygulamasında katılım belgesi oluşturulması gerektiğini düşünelim. Fonksiyon, kendisine iletilen kayıt kimliğini kullanarak gerekli bilgileri alır, belgeyi oluşturur ve sonucu kalıcı depoya yazar.

Bu tasarımda geliştirici, belge işinin yürütüleceği sunucuyu tek tek seçmek yerine fonksiyonun girdisini, işlemini ve erişeceği hizmetleri tanımlar. Ancak yanlış kayıt için belge oluşturulmaması veya aynı iş tekrar geldiğinde ikinci bir sonuç üretilmemesi gibi kurallar uygulama tarafından korunmalıdır.

Sorumlulukların paylaşılması

FaaS, altyapı yönetimini azaltır; uygulama sorumluluğunu ortadan kaldırmaz. Genel ayrım Tablo 8'de gösterilmiştir. Ayrıntılı sınırlar, seçilen hizmete ve çalıştırma planına göre değişebilir.

Tablo 8. Yönetilen FaaS modelinde temel sorumlulukların paylaşılması.

Alan	Bulut Platformunun Temel Görevi	Uygulama Ekibinin Temel Görevi
Çalışma ortamı	Altyapıyı sağlamak ve işletmek	Desteklenen ortamı ve yapılandırmayı seçmek
Çağrıların yürütülmesi	Tanımlı tetikleme mekanizmasıyla kodu çalıştırmak	Tetikleyiciyi ve girdinin işlenmesini tanımlamak
Kapasite	Hizmetin sunduğu modele göre kapasiteyi yönetmek	Sınırları ve bağımlı hizmetlerin kapasitesini değerlendirmek
İşlem doğruluğu	Çalıştırma olanaklarını sunmak	Doğrulama, iş kuralları ve tekrar işleme davranışını tasarlamak
Veri ve erişim	Veri ve erişim hizmetlerini sunmak	Gerekli veriyi, izinleri ve kalıcılığı yapılandırmak

Örneğin platform yeni fonksiyon örnekleri çalıştırabilse bile, bu örneklerin eriştiği veritabanı aynı artışı karşılayamayabilir. Otomatik kapasite yönetimi, bütün uygulamanın sınırsız ölçeklenmesi anlamına gelmez.

Mikroservislerle ilişkisi

Mikroservis mimarisi, iş yeteneklerinin nasıl ayrıldığını; FaaS ise kodun nasıl çalıştırıldığını açıklar. Bir mikroservis fonksiyonlarla gerçekleştirilebilir veya konteyner hizmeti olarak çalışabilir. Aynı uygulama her iki modeli birlikte kullanabilir.

Bu nedenle bir uygulamadaki her metodu ayrı fonksiyona dönüştürmek gerekli değildir. Çalıştırma birimi, anlamlı bir iş sorumluluğuna göre belirlenmeli; fonksiyonun çağrılması, durumu ve bağımlılıkları birlikte değerlendirilmelidir.

5.2. Tetikleyiciler ve Fonksiyonun Çalışma Akışı

Bir fonksiyonun kodunun hazırlanması, hangi durumda çalıştırılacağını tek başına belirlemez. *Trigger (tetikleyici)*, fonksiyonun çağrılmasını başlatan isteği, olayı veya zamanlama koşulunu tanımlar. Tetikleme mekanizması, gerekli girdiyi fonksiyona iletir. Örneğin kuyruk mesajının içeriği veya yüklenen dosyanın kimliği fonksiyonun girdisi olabilir. [27]

Temel tetikleyici türleri

Fonksiyonlar farklı ihtiyaçlara göre tetiklenebilir:

- **HTTP isteği:** İstemci bir uç noktaya istek gönderir. Fonksiyon girdiyi işler ve yanıt üretir.
- **Kuyruk mesajı:** İşlenmek üzere gelen mesaj, fonksiyonun çağrılmasını sağlar.
- **Depolama olayı:** Dosya yüklenmesi gibi bir değişiklik, ilgili işlemi başlatır.
- **Zamanlanmış tetikleme:** Belirlenen zamanlarda veya aralıklarla bir iş yürütülür.

Bu seçeneklerin desteği ve yapılandırılması platforma göre değişir. Tetikleyici, yalnızca kodun başlatılmasını sağlar; girdinin geçerliliği ve yapılacak işlemin kuralları fonksiyon tarafından değerlendirilmelidir.

Örneğin etkinlik afişi yüklendiğinde görüntü boyutlarını denetleyen bir fonksiyon çalıştırılabilir. Burada tetikleyici dosyanın yüklenmesidir; dosyanın kabul edilen biçim ve boyutta olup olmadığını belirlemek ise uygulamanın görevidir.

Fonksiyonun yürütülmesi

Bir çağrının temel akışı şu şekilde açıklanabilir:

1. İstek veya olay, yapılandırılmış tetikleme mekanizmasına ulaşır.
2. Platform, fonksiyonu uygun bir çalışma ortamında çağırır.
3. Fonksiyon girdiyi doğrular ve gerekli iş kurallarını uygular.
4. İhtiyaç duyduğu hizmetlere erişerek işlemi gerçekleştirir.
5. Gerekli sonucu kalıcı olarak saklar ve çağrı türüne uygun biçimde tamamlanır.

Şekil 3, bir isteğin veya olayın fonksiyonu başlatmasını ve fonksiyonun kalıcı verilerle etkileşimini göstermektedir. HTTP çağrısında sonuç doğrudan istemciye döndürülebilir; mesaj veya olay tetiklemesinde ise sonuç daha sonra sorgulanmak üzere saklanabilir.

Platform mevcut bir çalışma ortamını yeniden kullanabilir veya yeni bir ortam hazırlayabilir. Bu nedenle her çağrının mutlaka yeni bir sunucu ya da süreç oluşturduğu düşünülmemelidir. Bununla birlikte, sonraki çağrının aynı ortamda yürütüleceğine de güvenilmemelidir. [28]

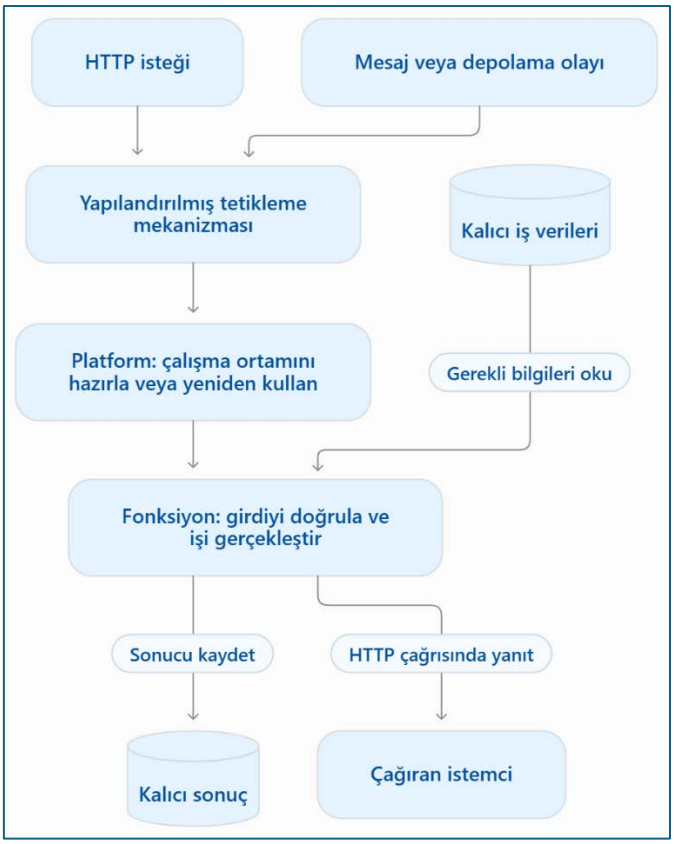
Yanıt ile kalıcı sonuç arasındaki fark

HTTP ile çağrılan bir fonksiyon, sonucu doğrudan istemciye döndürebilir. Kuyruk veya depolama olayıyla tetiklenen fonksiyonda ise aynı biçimde yanıt bekleyen bir kullanıcı bağlantısı bulunmayabilir. Sonuç bir veri deposuna yazılabilir veya başka bir mesajla ilgili bileşene bildirilebilir.

Örneğin katılım belgesi oluşturan fonksiyon dosyayı nesne depolamada saklar ve belge adresini işlem kaydına yazar. Kullanıcı daha sonra bu kaydı sorgulayarak belgeye ulaşır.

Fonksiyonun dönüş değeri ile uygulamanın saklaması gereken iş sonucu her zaman aynı şey değildir.

Fonksiyonun gerekli veri hizmetlerine erişebilmesi için bağlantı ve izin yapılandırmaları da sağlanmalıdır. Bir tetikleyicinin tanımlanması, diğer hizmetlere erişim yetkisinin kendiliğinden verildiği anlamına gelmez.



Şekil 3. Tetiklenen fonksiyonun çalıştırılması ve kalıcı verilerle etkileşimi.

HTTP ve olay yolları, alternatif tetikleme örnekleridir. Şema, her fonksiyonun bütün tetikleyicilere aynı anda bağlı olduğunu göstermez.

Harici durum ve tekrar çağrı

Fonksiyon, doğru çalışmak için önceki çağrıdan bellekte kalan bilgiye bağımlı olmamalıdır. Belge oluşturulduğu bilgisi yalnızca bellekte tutulursa yeni çalışma ortamı bu sonucu bilemez. Gerekli durum harici ve uygun kalıcılığa sahip bir hizmette korunmalıdır.

Çalışma ortamında geçici önbellek veya yeniden kullanılabilir bağlantılar bulunabilir. Ancak bunların kaybolması, işin doğruluğunu bozmamalıdır. Ayrıca olaylar tekrar teslim edilebileceğinden fonksiyonun idempotent davranması gerekebilir. [28]

Belge örneğinde aynı kayıt için ikinci çağrı geldiğinde fonksiyon mevcut sonucu tanıyabilir. Birden fazla çağrı aynı anda çalışabileceğinden, bu koruma yalnızca “önce kontrol et” adımına bırakılmamalı; kalıcı kayıttaki benzersizlik ve işlem kurallarıyla desteklenmelidir.

Hata durumundaki yeniden deneme ve teslim davranışı ise tetikleyici türüne göre değerlendirilir. HTTP çağrısında istemci tekrar deneyebilir; mesaj veya olay tetiklemesinde platformun ilgili mekanizması devreye girebilir. Bütün tetikleyicilerin aynı hata davranışına sahip olduğu varsayılmamalıdır.

5.3. Başlangıç Gecikmesi, Sınırlar ve Model Seçimi

Fonksiyon tabanlı çalıştırma, altyapı yönetimini azaltırken belirli çalışma koşulları getirir. Bir işin FaaS için uygunluğu değerlendirilirken yalnızca kodun çalışması değil; "başlangıç gecikmesi, çalışma sınırları ve bağımlı hizmetlere etkisi" de dikkate alınmalıdır.

Soğuk başlatma

Cold start (soğuk başlatma), bir çağrıyı karşılamak için çalışma ortamının hazırlanması gerektiğinde oluşan ek başlangıç gecikmesidir. Çalışma ortamının başlatılması, gerekli kütüphanelerin yüklenmesi ve başlangıç kodunun yürütülmesi bu süreye katkıda bulunabilir. Hazır ortam yeniden kullanıldığında aynı hazırlık adımlarının tamamı tekrarlanmayabilir. [29]

Soğuk başlatma her çağrıda gerçekleşmez. Ancak yeni ortam gerektiğinde, örneğin yük artışı sırasında, görülebilir. Etkisi kullanılan platforma, çalışma ortamına ve uygulamanın başlangıç işlemlerine bağlıdır.

Etkinlik uygulamasında arka planda belge oluşturulması için küçük bir başlangıç gecikmesi kabul edilebilir. Kullanıcının doğrudan beklediği bir sorguda ise aynı gecikme daha belirgin hissedilebilir. Bu nedenle seçim, yalnızca ortalama yanıt süresine değil, gecikmenin kullanıcı ve iş açısından önemine de dayanmalıdır.

Bazı platformlar hazır kapasite tutma seçenekleri sunar. Bunlar başlangıç gecikmesini azaltabilir; ancak ek maliyet oluşturabilir. [29]

Çalışma sınırları

FaaS hizmetlerinde süre, bellek, işlem kapasitesi, girdi boyutu ve eşzamanlı çağrı sayısı gibi sınırlar bulunabilir. Bu sınırlar sağlayıcıya, tetikleyiciye ve çalıştırma planına göre değişir. Örneğin Azure Functions'ın farklı barındırma seçenekleri, çalışma süresi ve ölçekleme bakımından farklı koşullar sunar. [30]

Bir iş seçilen sınırları aşıyorsa daha fazla kaynak ayırmak, işi uygun parçalara bölmek veya farklı bir çalıştırma modeli kullanmak gerekebilir. Ancak işi bölmek, ara sonuçların korunması ve başarısız adımların sürdürülmesi gibi yeni sorumluluklar getirir.

Uzun süren bir işin bütün FaaS platformlarında çalışamayacağı söylenemez. Önemli olan, seçilen hizmetin ilgili iş yükünü hangi koşullarda desteklediğini incelemektir.

Eşzamanlılık ve bağımlı hizmetler

Platform, yük arttığında daha fazla çağrıyı aynı anda çalıştırabilir. Fakat fonksiyonların eriştiği veritabanı veya dış hizmet aynı kapasiteye sahip olmayabilir.

Örneğin çok sayıda belge fonksiyonu aynı anda çalışırsa her biri kayıt veritabanına bağlantı açabilir. Fonksiyon kapasitesinin artması, veritabanındaki bağlantı sınırına ulaşılmasına neden olabilir. Bu nedenle eşzamanlılık ve örnek sayısı, bağımlı hizmetlerin kapasitesiyle birlikte değerlendirilmelidir. Yönetilen konteyner platformlarında da örnek sayısını sınırlama gibi yapılandırmalar bu amaçla kullanılabilir. [31]

FaaS veya konteyner seçimi

Tablo 9, çalıştırma modeli seçimi için temel değerlendirme noktalarını göstermektedir. Bunlar kesin ürün kuralları değil, iş yükünü incelemek için başlangıç ölçütleridir.

Tablo 9. FaaS ve konteyner tabanlı çalıştırma için seçim ölçütleri.

İş Yükü veya Gereksinim	FaaS Açısından Değerlendirme	Konteyner Açısından Değerlendirme
Kısa ve olayla başlayan iş	Doğal bir adaydır	Ayrı işçi veya görev olarak çalıştırılabilir
Başlangıç gecikmesine duyarlılık	Hazır kapasite seçenekleri incelenir	Hazır örneklerin tutulması değerlendirilebilir
Özel sistem bağımlılıkları	Platformun paketleme ve ortam desteği kontrol edilir	İmaj üzerinde daha fazla denetim sağlanabilir
Uzun süreç veya bağlantı	Süre ve yaşam döngüsü koşulları incelenir	Sürekli çalışma ihtiyacına uygun platform seçilir
Kalıcı iş verisi	Çalışma ortamından bağımsız korunmalıdır	Konteynerin yaşamından bağımsız korunmalıdır

Konteyner tabanlı çalıştırma da serverless bir platformda sunulabilir. Bu nedenle seçim, “serverless mı, konteyner mi?” şeklinde mutlak bir karşıtlık değildir; fonksiyon olarak sunulan kod ile konteyner hizmeti olarak sunulan uygulamanın hangi ortamda yürütüleceği değerlendirilir.

Örneğin yüklenen afişin biçimini doğrulama işi FaaS için uygun olabilir. Özel kütüphaneler kullanan ve sürekli iş alan bir bileşende ise konteyner tabanlı model tercih edilebilir. Aynı uygulamanın farklı bileşenleri için farklı seçimler yapılabilir.

Maliyet de kullanım biçimine bağlıdır. Çağrı sıklığı, çalışma süresi, ayrılan kaynaklar, hazır kapasite ve veri aktarımı birlikte düşünülmelidir. Serverless kullanılması, her iş yükünde en düşük maliyetin elde edileceği anlamına gelmez.

6. Bulut Sağlayıcı Örnekleri ve Bütünleşik Uygulama Senaryosu

Bu bölümde hizmet arayüzleri, mesajlaşma ve fonksiyon tabanlı çalıştırmanın bulut sağlayıcılarındaki karşılıkları ele alınacaktır. Örnekler, ürün adlarını ezberlemek yerine aynı mimari ihtiyacın farklı hizmetlerle nasıl karşılanabileceğini gösterecektir.

Ardından etkinlik kayıt senaryosu üzerinden hizmet sınırları, veri sahipliği ve iletişim yöntemleri bir araya getirilecektir. Yerel uygulamada kayıt hizmeti, bildirim hizmeti ve mesaj aracısı Docker Compose ile birlikte çalıştırılacaktır. Böylece konteyner bilgisi pekiştirilirken, bir kaydın oluşturulması ile ilgili bildirimin tamamlanması arasındaki fark gözlemlenebilecektir.

6.1. AWS, Azure ve Google Cloud Kavram Eşleştirmesi

Önceki bölümlerde ele alınan API erişimi, mesajlaşma ve fonksiyon çalıştırma ihtiyaçları, bulut sağlayıcılarının yönetilen hizmetleriyle karşılanabilir. AWS, Microsoft Azure ve Google Cloud benzer mimari amaçlara yönelik hizmetler sunar. Ancak bu hizmetlerin özellikleri, teslim güvenceleri ve çalıştırma koşulları bire bir aynı değildir.

Tablo 10, kavramları somutlaştırmak için örnek hizmetleri göstermektedir.

Tablo 10. Temel mimari ihtiyaçların bulut sağlayıcılarındaki örnek karşılıkları.

Mimari İhtiyaç	AWS	Microsoft Azure	Google Cloud
API giriş katmanı	Amazon API Gateway	Azure API Management	API Gateway
İşlerin tüketiciler arasında paylaşılması	Amazon SQS	Azure Service Bus kuyruğu	Pub/Sub içindeki aynı aboneliğe bağlı tüketiciler
Bağımsız aboneliklere mesaj dağıtımı	Amazon SNS; gerektiğinde SQS kuyruklarıyla birlikte	Azure Service Bus konusu ve abonelikleri	Pub/Sub konusu ve ayrı abonelikler
Fonksiyon çalıştırma	AWS Lambda	Azure Functions	Cloud Run functions

Tablodaki API hizmetleri [32]–[34], mesajlaşma örnekleri [21], [35]–[37] ve fonksiyon hizmetleri [25], [26], [38] resmî belgelerde açıklanmaktadır.

API erişimi

Üç sağlayıcıdaki API giriş hizmetleri, istemci isteklerinin arka uçlara yönlendirilmesine yardımcı olur. Arka uç, konteynerde çalışan kayıt hizmeti veya bir fonksiyon olabilir.

Örneğin `/kayitlar` isteği giriş katmanından kayıt işlevine iletilebilir. Bununla birlikte kontenjan kontrolü ve aynı katılımcının tekrar kaydolmasının önlenmesi, uygulamanın sorumluluğunda kalır. Azure API Management ayrıca API yönetimi ve geliştirici portalı gibi bileşenler içerir; bu nedenle tabloda aynı satırda bulunması, diğer ürünlerle aynı kapsamda olduğu anlamına gelmez. [33]

Mesajlaşmada model farkı

Amazon SQS — Simple Queue Service (basit kuyruk hizmeti), işlerin kuyruk üzerinden tüketicilere aktarılmasına örnektir. *Amazon SNS — Simple Notification Service (basit bildirim hizmeti)* ise yayınla–abone ol dağıtımı sağlar; bağımsız SQS kuyruklarına mesaj iletmek için de kullanılabilir. [21], [35]

Azure Service Bus, hem kuyruk hem de *topic (konu)* ve abonelik yapıları sunar. Kuyrukta tüketiciler işleri paylaşırken, konuya bağlı ayrı abonelikler ilgili mesajları bağımsız alabilir. [36]

Google Cloud Pub/Sub'da ise bir konuya bağlı aynı aboneliğin tüketicileri işleri paylaşır; farklı abonelikler yayımlanan mesajları ayrı ayrı alabilir. Dolayısıyla yalnızca ürün adına bakmak yerine konu, abonelik ve tüketici düzeni incelenmelidir. [37]

Fonksiyon çalıştırma

AWS Lambda, Azure Functions ve Cloud Run functions, istek veya olayla başlatılan kodun yönetilen ortamda çalıştırılmasına örnektir. Aynı belge oluşturma işlevi bu platformlarda gerçekleştirilebilir; ancak tetikleyici yapılandırması ve hizmet bağlantıları sağlayıcıya göre uyarlanır. [25], [26], [38]

Bu hizmetlere geçiş, önceki bölümlerdeki tasarım gereksinimlerini kaldırmaz. İş kimliği, kalıcı sonuç, tekrar işleme davranışı ve gerekli erişim izinleri yine belirlenmelidir. Sağlayıcı seçimi yapılırken ürün adından önce iletişim modeli, işlem doğruluğu ve iş yükünün çalışma koşulları değerlendirilir.

6.2. Etkinlik Kayıt Uygulamasının Gereksinimleri ve Mimarisi

Üniversitede düzenlenen etkinlikler için katılımcı kaydı alan küçük bir uygulama tasarlanacaktır. Kullanıcı etkinlikleri görüntüleyebilecek, seçtiği etkinliğe kaydolabilecek ve kayıt durumunu sorgulayabilecektir. Başarılı kayıt sonrasında bir bildirim işi oluşturulacak; bu iş ayrı bir hizmet tarafından yürütülecektir.

Senaryonun temel gereksinimi, kaydın oluşturulması ile bildirimin tamamlanmasının birbirinden ayrılmasıdır. Bildirim hizmetinin yavaşlaması veya geçici olarak durması, oluşturulmuş kaydı geçersiz kılmamalıdır.

İşlevler ve iş kuralları

Uygulama aşağıdaki davranışları sağlamalıdır:

- Etkinlikler listelenebilmeli ve etkinlik bilgileri sorgulanabilmelidir.
- Kayıt isteğindeki zorunlu alanlar doğrulanmalıdır.
- Etkinlik kayıt kabul etmeli ve boş kontenjan bulunmalıdır.
- Aynı katılımcı, aynı etkinliğe ikinci kez kaydedilmemelidir.

- Başarılı kayıt kalıcı olarak saklanmalı ve kayıt kimliğiyle sorgulanabilmelidir.
- Bildirim işi, kayıtla ilişkilendirilerek takip edilebilmelidir.
- Aynı bildirim işinin tekrar alınması, ikinci bir başarılı sonuç kaydı oluşturmamalıdır.

Bildirim, öğretim amaçlı olarak bir sonuç kaydı oluşturacak şekilde gerçekleştirilebilir. Gerçek bir e-posta veya kısa mesaj hizmetine bağlantı kurulması gerekli değildir.

Bileşenler ve veri sorumlulukları

Yerel uygulama; kayıt hizmeti, bildirim hizmeti ve mesaj aracısından oluşacaktır. Bu küçük örnekte etkinlik bilgileri de kayıt hizmeti içinde yönetilecektir. Docker Compose, bileşenlerin birlikte başlatılmasını ve aynı uygulama ağı üzerinden haberleşmesini sağlayacaktır.

Tablo 11, bileşenlerin görevlerini ve yönettikleri verileri göstermektedir.

Tablo 11. Etkinlik kayıt uygulamasının bileşenleri ve veri sorumlulukları.

Bileşen	Temel Görev	Yönettiği Veri
Kayıt hizmeti	REST arayüzünü sunmak, kayıt kurallarını uygulamak ve bildirim işi oluşturmak	Etkinlikler, katılımcı kayıtları ve iletilecek işler
Mesaj aracısı	Bildirim işlerini tüketiciye aktarmak	Bekleyen mesajlar ve teslim bilgileri
Bildirim hizmeti	Kuyruktan iş almak, bildirimi gerçekleştirmek ve tekrarları tanımak	İşlenmiş iş kimlikleri ve bildirim sonuçları

Kayıt ve bildirim hizmetlerinin kalıcı verileri kendi sorumluluklarında tutulacaktır. Bildirim hizmeti kayıt tablolarını doğrudan değiştirmeyecek; ihtiyaç duyduğu bilgiyi mesaj içeriğinden alacaktır. Konteynerlerin yeniden oluşturulması sırasında gerekli verilerin korunması için kalıcı depolama kullanılacaktır.

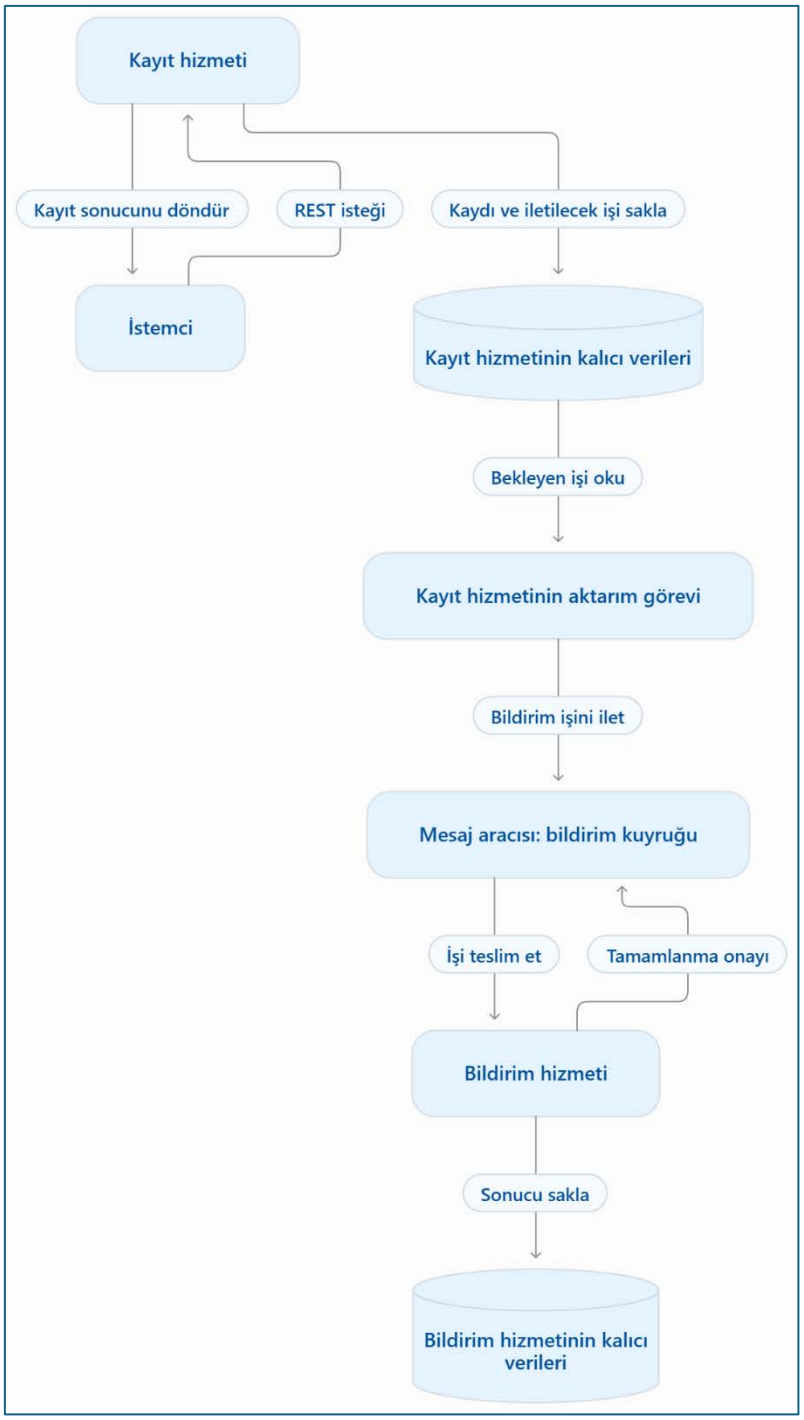
Kayıttan bildirime işlem akışı

Şekil 4, kullanıcı isteği ile arka plan bildirim işinin farklı yollarını göstermektedir. Kayıt hizmetinin kullanıcıya verdiği başarılı yanıt, kaydın oluşturulduğunu belirtir; bildirimin gönderildiğini belirtmez.

Kayıt ile iletilecek işin birlikte kalıcılaştırılması, kayıt tamamlandıktan sonra mesaj aktarımının başarısız olması durumunda işin unutulmasını önler. Şekildeki aktarım görevi, kayıt hizmeti içinde çalışan bir arka plan görevini temsil eder; ayrı bir konteyner olması gerekmez.

Bildirim hizmeti kapalıyken işler, yapılandırılmış kalıcılık ve saklama koşulları kapsamında kuyrukta bekler. Hizmet yeniden çalıştığında bu işleri alabilir. Tamamlanma onayından

önce kapanırsa aynı iş yeniden teslim edilebilir; bildirim hizmeti kalıcı iş kimliğini kullanarak mevcut sonucu tanımalıdır.



Şekil 4. Etkinlik kaydının oluşturulması ve bildirim işinin ayrı yürütülmesi.

Bulut mimarisiyle bağlantı

Docker Compose ile yerelde çalışan bu düzen, bulutta kullanılabilecek hizmet ayrımını somutlaştırır. Buluta taşındığında kayıt hizmeti yönetilen konteyner ortamında, kuyruk yönetilen mesajlaşma hizmetinde ve bildirim işi konteyner tüketicisi veya fonksiyon olarak çalıştırılabilir.

Çalıştırma ortamı değişse de kayıt kuralları, veri sahipliği, mesajın anlamı ve tekrar işleme gereksinimi korunur. Böylece uygulama, önceki haftaların konteyner bilgisini bu haftanın hizmet ve iletişim tasarımıyla birleştirir.

6.3. Aşamalı Yerel Uygulama

Yerel uygulamanın amacı, hazır etkinlik kayıt sistemini çalıştırarak hizmet arayüzlerini, kuyruk üzerinden iletişimi ve hata durumlarındaki davranışı gözlemlemektir. Öğrenci küçük bir değişiklik yaparak uygulamanın kurallarını da sınayacaktır.

Hafta 5'te kullanılan konteyner ortamı temel alınacaktır. Kayıt hizmeti, bildirim hizmeti ve mesaj aracısı Docker Compose ile birlikte çalıştırılacaktır. Kubernetes veya bulut hesabı gerekli değildir. Hazır uygulama kodları, ders notunun eklerinde yer alan uygulama kod paketinde sunulmaktadır. Kurulum, çalıştırma komutları ve deneylerin ayrıntılı adımları ise "Hafta 7 – Yerel Uygulama Yönergesi" başlıklı ekte açıklanmaktadır.

Birinci aşama: REST arayüzünü kullanma

Öğrenci etkinlikleri sorgular, bir kayıt oluşturur ve oluşturulan kaydı kimliğiyle görüntüler. Ardından eksik alan, bulunamayan etkinlik ve tekrar kayıt gibi istekleri deneyerek yanıtları karşılaştırır.

Bu aşamada amaç, yalnızca başarılı yanıt almak değil, isteğin hangi kurala göre kabul edildiğini veya reddedildiğini açıklayabilmektir. Öğrenci ayrıca bir doğrulama kuralını değiştirir ve değişikliğin sonucunu kontrol eder.

İkinci aşama: Kayıt ve bildirim sonuçlarını ayırma

Başarılı bir kayıt oluşturulduktan sonra kayıt verisi ile bildirim sonucu ayrı incelenir. Öğrenci, kayıt hizmetinin yanıtının bildirimin tamamlandığını göstermediğini doğrular.

Bildirim işinin kimliği ve ilgili kayıt kimliği kullanılarak iki işlem arasındaki ilişki takip edilir. Gerçek e-posta gönderimi yerine oluşturulan bildirim sonuç kaydı gözlemlenir.

Üçüncü aşama: Bildirim hizmetini durdurma

Bildirim hizmeti durdurulur; kayıt hizmeti ve mesaj aracısı çalışmaya devam eder. Bu sırada yeni kayıtlar oluşturulur.

Kayıtlar tamamlanırken bildirim işleri kuyrukta beklemelidir. Bildirim hizmeti yeniden başlatıldığında bekleyen işler alınarak sonuçlandırılmalıdır. Öğrenci, bu davranışın hizmetlerin tamamlanma zamanlarını birbirinden ayırdığını açıklar.

Dördüncü aşama: Aynı işi tekrar gönderme

Daha önce tamamlanmış bir bildirim işi, aynı iş kimliği korunarak yeniden iletilir. Bildirim hizmetinin mevcut sonucu tanıması ve ikinci bir başarılı sonuç kaydı oluşturmaması beklenir.

Yeni bir iş kimliğiyle gönderim, aynı mantıksal işin tekrarıyla karıştırılmamalıdır. Bu deneyde mesajın tekrar teslim edilmesi ile uygulamanın iş etkisi birbirinden ayrılır.

Beşinci aşama: Kalıcı veriyi kontrol etme

Hizmet konteynerleri, kalıcı depolama alanları korunarak yeniden oluşturulur. Önceden oluşturulan kayıtlar ve bildirim sonuçları tekrar sorgulanır. Böylece konteynerin yaşam süresi ile iş verisinin yaşam süresinin farklı olduğu gözlemlenir.

Bu deney sırasında kalıcı depolama alanlarının silinmesi, hizmetin yeniden oluşturulmasından farklı bir işlemdir.

Aşamalar ve beklenen gözlemler Tablo 12'de özetlenmiştir.

Tablo 12. Yerel uygulama deneyleri ve beklenen sonuçlar.

Deney	Beklenen Gözlem
Geçerli ve hatalı REST istekleri	Sözleşmeye uygun başarı ve hata yanıtları
Bildirim hizmeti kapalıyken kayıt oluşturma	Kaydın tamamlanması, bildirim işinin beklemesi
Bildirim hizmetini yeniden başlatma	Bekleyen işlerin işlenmesi
Aynı iş kimliğiyle tekrar gönderim	İkinci başarılı sonuç kaydının oluşmaması
Kalıcı depolamayı koruyarak konteynerleri yeniden oluşturma	Önceki iş verilerine erişimin sürmesi

Rapor ve teslim

Öğrenci, yerel uygulama çalışmasını kısa bir raporla belgelendirir. Raporda aşağıdaki bilgiler bulunmalıdır:

- **Bileşenler ve sorumlulukları:** Kayıt hizmeti, bildirim hizmeti ve mesaj aracısının görevleri; verilerin hangi bileşen tarafından yönetildiği.
- **Yapılan değişiklik:** Değiştirilen doğrulama kuralı veya küçük işlev ile değişikliğin sonucunu gösteren örnek.
- **Deney sonuçları:** Geçerli ve hatalı istekler, bildirim hizmetinin durdurulup yeniden başlatılması, aynı işin tekrar gönderilmesi ve kalıcılık deneyinin sonuçları.
- **Kısa değerlendirme:** Gözlemlerin hizmet ayrımı, eşzamansız iletişim, idempotentlik ve veri kalıcılığıyla ilişkisi.

Ekran görüntüleri veya istek–yanıt çıktıları, ilgili deneyin kısa açıklamasıyla birlikte verilmelidir. Yalnızca konteynerlerin çalıştığını gösteren görüntüler yeterli değildir.

Teslim paketi, PDF biçimindeki rapor ile öğrencinin değiştirdiği kaynak dosyalarını içerir. Hazır kod paketinin tamamının yeniden teslim edilmesi gerekmez. Dosya adları ve teslim yöntemi, **"Hafta 7 – Yerel Uygulama Yönergesi"** içinde belirtilir.

6.4. İsteğe Bağlı Bulut Uygulaması ve Değerlendirme

Yerel uygulamayı tamamlayan öğrenciler, bildirim işlevini bulutta fonksiyon olarak çalıştırarak çalışmalarını genişletebilir. Bu çalışma isteğe bağlıdır; ancak FaaS yaklaşımının gerçek bir yönetilen ortamda deneyimlenmesi açısından teşvik edilir. AWS Lambda, Azure Functions veya Google Cloud Run functions seçeneklerinden birinin kullanılması yeterlidir.

Amaç, yerel uygulamanın bütün bileşenlerini buluta taşımak değil, belirli bir işlevin fonksiyon olarak dağıtılması ve tetikleyiciyle çalıştırılmasıdır.

Bulut çalışmasının kapsamı

Öğrenci, hazır bildirim işlevini seçtiği platformun fonksiyon giriş yapısına uyarlar. İş kimliği ve kayıt bilgilerini içeren bir HTTP isteğiyle fonksiyonu çağırır. Fonksiyon girdiyi doğrular, bildirim sonucunu üretir ve sonucu uygun bir kalıcı veri hizmetinde saklar.

Bu temel genişletmede mesaj kuyruğunun buluta taşınması zorunlu değildir. HTTP tetikleyicisinin kullanılması, fonksiyon dağıtımını ve çağrısını daha sınırlı bir çalışma içinde deneyimlemeyi sağlar. İsteyen öğrenciler ayrıca yönetilen kuyruk veya abonelik bağlantısı kurarak mesajla tetiklemeyi deneyebilir.

Çalışmada aşağıdaki noktalar doğrulanmalıdır:

- Fonksiyonun tetikleyicisi ve gerekli erişim izinleri yapılandırılmıştır.
- Geçerli girdiler işlenmekte, eksik veya hatalı girdiler reddedilmektedir.
- İş sonucu, fonksiyonun geçici belleğinden bağımsız korunmaktadır.
- Aynı iş kimliğiyle tekrar çağrı, ikinci bir başarılı sonuç kaydı oluşturmamaktadır.

Yalnızca fonksiyon kodunu yüklemek veya platformun test ekranında çalıştırmak, dışarıdan tetikleme bağlantısının kurulduğunu göstermeyebilir. Kullanılan çağrı yolu ve sonuç açıkça doğrulanmalıdır.

Rapor ve ek kanıtlar

Bulut çalışmasını yapan öğrenci, yerel uygulama raporuna kısa bir “Bulut Genişletmesi” kısmı ekler. Bu kısımda:

- Seçilen sağlayıcı ve kullanılan hizmetler.
- Tetikleyici türü ve işlem akışı.
- Başarılı çağrı ile kalıcı sonucu gösteren çıktılar.
- Hatalı girdi ve aynı işin tekrar çağrılması deneyleri.
- Yerel uygulamaya göre yapılan kod veya yapılandırma değişiklikleri.

yer almalıdır. Parola, erişim anahtarı ve bağlantı bilgileri teslim dosyalarında paylaşılmamalıdır. Deney sonrasında gereksiz kaynaklar kaldırılmalı veya kapatılmalıdır.

Değerlendirme

Yerel çalışma, bütün öğrenciler için temel değerlendirmeyi oluşturur. Bulut genişletmesi ise ayrıca değerlendirilir; yerel deneylerin yerine geçmez.

Temel değerlendirmede REST arayüzünün kullanılması, yapılan küçük değişiklik, kuyruk davranışı, tekrar işleme ve veri kalıcılığının doğru açıklanması esas alınır. Bulut genişletmesinde ise fonksiyonun gerçek tetikleyiciyle çalışması, sonucun korunması ve yapılandırmanın gerekçelendirilmesi önemlidir. Kullanılan ürün sayısı veya sağlayıcı tercihi tek başına başarı ölçütü değildir.

7. Bölüm Özeti

Bu hafta, uygulamaların bulut ortamının esneklik ve yönetilebilirlik olanaklarından yararlanacak biçimde tasarlanması ele alınmıştır. Bir uygulamanın bulutta veya konteynerde çalışması, tek başına buluta uygun mimariye sahip olduğunu göstermez. Uygulamanın hizmet sınırları, veri sahipliği, iletişim yöntemleri ve çalışma örneklerinin değişmesine karşı davranışı birlikte değerlendirilmelidir.

Hizmet odaklı mimari, iş yeteneklerinin tanımlı arayüzler ve teknik sözleşmeler üzerinden kullanılmasını sağlar. Monolitik uygulama, modüler monolit ve mikroservisler; dağıtım, ölçekleme ve yönetim açısından farklı seçenekler sunar. Mikroservisler bağımsız geliştirme ve çalıştırmayı destekleyebilir; ancak ağ iletişimi ve veri tutarlılığı gibi ek sorumluluklar getirir. Hizmet sınırlarının iş yeteneklerine göre belirlenmesi ve verilerin sorumlu hizmet tarafından yönetilmesi, bu bağımlılıkların sınırlandırılmasına yardımcı olur.

Durumsuz çalışma örneklerinde gerekli iş verisi, örneğin geçici belleğinden bağımsız korunur. Böylece farklı örnekler isteklere hizmet verebilir ve bir örneğin yeniden oluşturulması kalıcı verinin kaybolmasına neden olmaz. REST arayüzleri kaynakları, işlem yöntemlerini ve yanıtları düzenlerken; API ağ geçidi, hizmet keşfi ve yük dengeleme, isteklerin uygun hizmet ve çalışma örneğine ulaşmasını sağlar.

Eşzamanlı iletişimde iş akışı karşı hizmetin yanıtını bekler; eşzamansız iletişimde işin iletilmesi ile tamamlanması ayrılabilir. İş kuyruğu aynı tür işleri tüketiciler arasında paylaştırır, yayınla–abone ol modeli ise mesajları bağımsız aboneliklere dağıtır. Olay güdümlü tasarımda gerçekleşmiş değişiklikler diğer hizmetlerin işlerini başlatabilir. Bununla birlikte zaman aşımı, tekrar teslim ve başarısız işler dikkate alınmalıdır. İdempotentlik tekrarların ek iş etkisi oluşturmasını önlerken, nihai tutarlılık farklı görünümlerin gecikmeli güncellenmesini açıklar. Başarısız mesaj kuyruğu ve işlemsel

çıkış kutusu gibi yaklaşımlar, işlerin kaybolmadan takip edilmesine ve sürdürülmesine yardımcı olur.

Serverless yaklaşım altyapı yönetiminin önemli bölümünü platforma bırakır; FaaS ise belirli işlevlerin istek veya olaylarla çalıştırılmasını sağlar. Uygulamanın doğruluğu, kalıcı durum ve erişim izinleri yine uygulama ekibinin sorumluluğundadır. Fonksiyon veya konteyner seçimi; başlangıç gecikmesi, çalışma süresi, kaynak ihtiyacı ve bağımlı hizmetlerin kapasitesine göre yapılır. Etkinlik kayıt uygulaması bu kararları bir araya getirerek, kaydın oluşturulması ile bildirimin tamamlanmasını ayırır ve konteyner bilgisini hizmetler arası iletişimle ilişkilendirir.

Kaynakça

[1] "toc/DEFINITION.md at main · cncf/toc," GitHub. Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/cncf/toc/blob/main/DEFINITION.md
[2] "Reference Model for Service Oriented Architecture v1.0." Accessed: Oct. 13, 2026. [Online]. Available: https://docs.oasis-open.org/soa-rm/v1.0/soa-rm.html
[3] ardalis, "Common web application architectures - .NET." Accessed: Oct. 13, 2026. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures
[4] claytonsiemens77, "Microservices Architecture Style - Azure Architecture Center." Accessed: Oct. 13, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices
[5] claytonsiemens77, "Use Domain Analysis to Model Microservices - Azure Architecture Center." Accessed: Oct. 13, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis
[6] claytonsiemens77, "Data Considerations for Microservices - Azure Architecture Center." Accessed: Oct. 13, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/microservices/design/data-considerations
[7] "REL05-BP06 Make systems stateless where possible - AWS Well-Architected Framework." Accessed: Oct. 13, 2026. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/framework/rel_mitigate_interaction_failure_stateless.html
[8] "Fielding Dissertation: CHAPTER 5: Representational State Transfer (REST)." Accessed: Oct. 13, 2026. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
[9] R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," Internet Engineering Task Force, Request for Comments RFC 9110, Jun. 2022. doi: 10.17487/RFC9110.
[10] claytonsiemens77, "Asynchronous Request-Reply Pattern - Azure Architecture Center." Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/asynchronous-request-reply

[11] claytonsiemens77, “API gateways - Azure Architecture Center.” Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/microservices/design/gateway
[12] “Service,” Kubernetes. Accessed: Oct. 14, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/service/
[13] “The Istio service mesh,” Istio. Accessed: Oct. 14, 2026. [Online]. Available: https://istio.io/latest/about/service-mesh/
[14] claytonsiemens77, “Interservice communication in microservices - Azure Architecture Center.” Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/microservices/design/interservice-communication
[15] claytonsiemens77, “Queue-Based Load Leveling Pattern - Azure Architecture Center.” Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling
[16] claytonsiemens77, “Competing Consumers Pattern - Azure Architecture Center.” Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/competing-consumers
[17] claytonsiemens77, “Publisher-Subscriber Pattern - Azure Architecture Center.” Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/publisher-subscriber
[18] claytonsiemens77, “Event-Driven Architecture Style - Azure Architecture Center.” Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven
[19] “spec/cloudevents/spec.md at main · cloudevents/spec,” GitHub. Accessed: Oct. 14, 2026. [Online]. Available: https://github.com/cloudevents/spec/blob/main/cloudevents/spec.md
[20] claytonsiemens77, “Retry pattern - Azure Architecture Center.” Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/retry
[21] “Amazon SQS at-least-once delivery - Amazon Simple Queue Service.” Accessed: Oct. 14, 2026. [Online]. Available: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/standard-queues-at-least-once-delivery.html
[22] spelluru, “Service Bus Dead-Letter Queues - Azure Service Bus.” Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-dead-letter-queues
[23] “Transactional outbox pattern - AWS Prescriptive Guidance.” Accessed: Oct. 14, 2026. [Online]. Available: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html
[24] “What is Cloud Run,” Google Cloud Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.cloud.google.com/run/docs/overview/what-is-cloud-run
[25] ggailey777, “Azure Functions Overview.” Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-functions/functions-overview

[26] "What is AWS Lambda? - AWS Lambda." Accessed: Oct. 14, 2026. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html
[27] ggailey777, "Triggers and Bindings in Azure Functions." Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-functions/functions-triggers-bindings
[28] "Best practices for working with AWS Lambda Event Functions - AWS Lambda." Accessed: Oct. 14, 2026. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html
[29] "Understanding the Lambda execution environment lifecycle - AWS Lambda." Accessed: Oct. 14, 2026. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/lambda-runtime-environment.html
[30] ggailey777, "Azure Functions Scale and Hosting." Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-functions/functions-scale
[31] "About instance autoscaling in Cloud Run services," Google Cloud Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.cloud.google.com/run/docs/about-instance-autoscaling
[32] "What is Amazon API Gateway? - Amazon API Gateway." Accessed: Oct. 14, 2026. [Online]. Available: https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html
[33] PatAltimore, "Azure API Management - Overview and Key Concepts." Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/api-management/api-management-key-concepts
[34] "About API Gateway," Google Cloud Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.cloud.google.com/api-gateway/docs/about-api-gateway
[35] "What is Amazon SNS? - Amazon Simple Notification Service." Accessed: Oct. 14, 2026. [Online]. Available: https://docs.aws.amazon.com/sns/latest/dg/welcome.html
[36] spelluru, "Introduction to Azure Service Bus Messaging - Azure Service Bus." Accessed: Oct. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-messaging-overview
[37] "What is Pub/Sub?," Google Cloud Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.cloud.google.com/pubsub/docs/overview
[38] "Cloud Run functions overview," Google Cloud Documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.cloud.google.com/run/docs/functions/overview

---

Hafta 7 – Çalışma Soruları

Soruları ders notuna bakmadan yanıtlamaya çalışınız. Yanıtlarınızda yalnızca kavramların tanımlarını değil, ilgili mekanizmanın neden ve nasıl kullanıldığını da açıklayınız. Komut veya yapılandırma dosyası yazmanız ve belirli bir bulut sağlayıcısının ürün adlarını ezberlemeniz gerekli değildir.

A. Hizmet Yaklaşımı ve Mimari Seçim

1. Bir bulut uygulamasında hizmet sınırı ve hizmet sözleşmesi neyi tanımlar? Bir hizmetin sözleşmesinde bulunması gereken en az dört unsuru açıklayınız. Hizmetin iç işleyişi değiştiğinde, onu kullanan bütün bileşenlerin de değiştirilmesi gerekir mi? Yanıtınızı sözleşmenin korunması açısından gerekçelendiriniz.
2. SOA ile SOAP arasındaki ilişkiyi açıklayınız. SOA yaklaşımıyla tasarlanan bir sistemin mutlaka SOAP kullanması gerekir mi? REST bu ilişkide hangi düzeyde bir kavramdır?
3. Küçük bir ekip tarafından geliştirilen ve düşük trafik alan bir uygulamanın bulutta çalıştırılması planlanmaktadır. Uygulamayı mikroservislere ayırmak her zaman doğru seçim midir? Monolitik yapı ile mikroservis yaklaşımını dağıtım, ölçekleme ve işletim yükü bakımından karşılaştırınız. Mikroservislere geçişi anlamlı kılabilecek iki gereksinim belirtiniz.

B. REST Arayüzleri ve Durum Yönetimi

4. Bir bulut depolama uygulaması; klasörleri listeleme, yeni klasör oluşturma ve belirli bir klasörün bilgilerini sorgulama işlemlerini sunacaktır. Bu işlemler için kaynak odaklı adresler ve uygun HTTP yöntemleri öneriniz. Başarılı oluşturma, geçersiz istek ve bulunamayan kaynak durumlarını hangi yanıtlarla ayırt edersiniz? Bir arayüzün HTTP üzerinden JSON döndürmesi, tek başına REST ilkelerine uygun olduğunu gösterir mi?
5. Stateless bir hizmetin hiç veri tutmadığı söylenebilir mi? Bir alışveriş uygulamasının birden fazla hizmet kopyası yük dengeleyici arkasında çalışmaktadır. Kullanıcının sepeti yalnızca isteği ilk karşılayan kopyanın belleğinde tutulursa hangi sorunlar ortaya çıkabilir? Oturum durumu ile kalıcı iş verisini ayırarak uygun bir yaklaşım açıklayınız.
6. Bir hava durumu hizmeti, yanıtındaki sıcaklık alanının adını değiştirmiş ve sayısal değer yerine metin döndürmeye başlamıştır. Hizmet kendi başına çalışmasına rağmen bazı istemciler hata vermektedir. Bu durumu hizmet sözleşmesi açısından açıklayınız. Bağımsız dağıtımı desteklemek için değişiklik nasıl yönetilebilir?

C. Mikroservis Sınırları ve Erişim

7. Bir çevrim içi eğitim platformunda ders içeriği ve sınav değerlendirme ayrı hizmetler tarafından yönetilmektedir. Sınav hizmetinin, ders içeriği hizmetine ait tablolara doğrudan yazması hangi bağımlılıkları doğurabilir? Veri sahipliği ve hizmet arayüzleri açısından daha uygun bir etkileşim yaklaşımı öneriniz. Ayrı veri sahipliği, mutlaka ayrı fiziksel veritabanı sunucuları gerektirir mi?

8. Bir bulut uygulamasındaki hizmet örneklerinin adresleri, ölçekleme ve yeniden oluşturma sırasında değişmektedir. API gateway ile service discovery bu ortamda hangi farklı sorunları çözer? İstemcilerin erişimi ile hizmetler arası erişim üzerinden açıklayınız. Bir hizmet adresinin bulunabilmesi, hizmetin kullanıma hazır ve işlevsel olarak doğru olduğunu kanıtlar mı?

9. Konteyner teknolojisi, Kubernetes ve mikroservis mimarisi hangi farklı düzeylerde kararlar ve olanaklar sunar? Bir uygulamayı birkaç konteynere bölmek, tek başına mikroservis mimarisi oluşturulduğunu gösterir mi? Orkestrasyon platformunun üstlenmediği iki uygulama sorumluluğunu açıklayınız.

D. İletişim ve Hata Dayanıklılığı

10. Bir seyahat uygulamasında kullanıcının seçtiği seferin güncel fiyatı sorgulanmakta, ayrıca geçmiş yolculuklara ilişkin ayrıntılı bir rapor hazırlanmaktadır. Bu iki işlem için eşzamanlı veya eşzamansız iletişimden hangisini tercih edersiniz? Kullanıcının anlık sonuca ihtiyacı, işlem süresi ve bağımlı hizmetin kesintisi bakımından seçimlerinizi gerekçelendiriniz.

11. Bir görüntü işleme sisteminde, yüklenen görsellerin küçük önizlemeleri birden fazla çalışan arasında paylaştırılacaktır. Ayrı bir gereksinim olarak her yeni görselin hem içerik denetimi hem arşivleme hizmeti tarafından işlenmesi istenmektedir. Work queue ve publish/subscribe yaklaşımlarını bu iki gereksinimle eşleştiriniz. Aynı kuyruğa daha fazla tüketici eklemek hangi durumda yükü paylaştırır, hangi gereksinimi tek başına karşılamaz?

12. Bir bulut uygulaması, bağımlı bir hizmetten zamanında yanıt alamamaktadır. Timeout, retry ve circuit breaker hangi farklı amaçlara hizmet eder? Zaman aşımı, karşı taraftaki işlemin yapılmadığını kesin olarak gösterir mi? Tekrar denemenin çift iş etkisi oluşturabileceği bir örnek vererek idempotent işlem tasarımının rolünü açıklayınız.

E. Olaylar ve Güvenilir İşleme

13. Bir stok yönetim sisteminde “StokSeviyesiDüştü” olayı ile “TedarikTalebiOluştur” komutu kullanılmaktadır. Bunların anlamları ve alıcılardan beklenen davranışlar nasıl farklılaşır? Depodaki değişikliğin raporlama ekranına kısa bir gecikmeyle yansımasını nihai tutarlılık açısından açıklayınız. Bu gecikme hangi koşullarda normal kabul edilebilir, hangi koşullarda araştırılmalıdır?

14. Bir ölçüm işleme hizmeti, kuyruktan aldığı ölçümü kalıcı sonuç deposuna yazmış; ancak mesaj onayını iletmeden kapanmıştır. Mesaj yeniden teslim edilirse ikinci bir sonuç oluşması nasıl önlenebilir? Tekrar kontrolünün yalnızca hizmet belleğinde tutulması neden yeterli değildir? Mesajı sonuç yazılmadan onaylamak hangi farklı riski doğurur?

15. Bir teknik destek sisteminde destek talebi veritabanına başarıyla yazılmış, ancak talebi sınıflandırma hizmetine iletecek mesaj gönderilememiştir. Talebin oluşturulması ile yapılacak iş arasındaki bağ nasıl korunabilir? Ayrıca sınıflandırılamayan, sürekli hata üreten bir mesajın normal işlerden ayrılması neden yararlıdır? Ayrı hata kuyruğuna alınması, sorunun çözüldüğü anlamına gelir mi?

F. Sunucusuz Çalışma ve Bütünleşik Tasarım

16. Serverless ve FaaS kavramlarını ilişkilendiriniz. Bir işlevin tetiklenmesi, çalışması ve ölçeklenmesi sırasında sağlayıcı ile geliştiricinin sorumlulukları nasıl paylaşılır? “Sunucusuz” ifadesi fiziksel sunucu olmadığı anlamına gelir mi? FaaS kullanılması, iş kuralları ve hata yönetimi sorumluluğunu ortadan kaldırır mı?

17. Belirli aralıklarla çalışan kısa bir veri dönüştürme işi ile kesintisiz bağlantı üzerinden uzun süreli oturum yöneten bir uygulamayı karşılaştırınız. Hangisi başlangıç düzeyinde FaaS için daha uygun bir örnektir? Seçiminizi tetiklenme biçimi, çalışma süresi ve durum gereksinimi üzerinden açıklayınız. Cold start ve geçici çalışma ortamı tasarımı nasıl etkileyebilir?

18. Bir üniversite, öğrencilerin yüklediği proje raporlarını işleyen bir bulut sistemi geliştirmektedir. Sistem, yüklenen dosyayı kalıcı olarak saklayacak, dosyadan metin çıkaracak ve küçük bir önizleme oluşturacaktır. Aşağıdaki gereksinimleri karşılayan bir tasarım öneriniz:
    - Dosya yükleme yanıtı, metin çıkarma ve önizleme işlemlerinin tamamlanmasını beklememelidir.
    - Metin çıkarma ve önizleme bileşenleri aynı dosya üzerinde birbirinden bağımsız çalışabilmelidir.
    - İşleme bileşenlerinden biri geçici olarak kullanılamadığında ilgili iş kaybolmamalıdır.
    - Aynı iş yeniden teslim edildiğinde ikinci bir sonuç kaydı oluşmamalıdır.
    - İşleme bileşenleri, kendi iş yüklerine göre ayrı ayrı ölçeklenebilmelidir.
    - Öğrenci, dosyanın yüklenmesi ile her bir işleme adımının tamamlanma durumunu ayrı ayrı görebilmelidir.

Hizmetlerin sorumluluklarını, kalıcı verilerin yerini ve mesajlaşma düzenini açıklayınız. İki işleme bileşeninin tek bir iş kuyruğundan mesaj alması bu gereksinimleri karşılar mı? İşleme adımlarının FaaS ile gerçekleştirilmesi hâlinde hangi tasarım sorumlulukları devam eder? Tasarımın doğruluğunu gösterecek en az dört kontrolü, beklenen sonuçlarıyla belirtiniz.